

RAPPORT FIL ROUGE

Plateforme ECOTRACK

IoT pour le pilotage de la collecte des déchets urbains

Candidats

POILLY Loris & FAUCHON Rémi

Cursus

Mastère INGETIS — M2 ITIS

Spécialité

Développement

Année

2025 — 2026 (M2)

Document

Rapport binôme couvrant les Blocs 1 à 4

Dépôt : juin 2026 — m.bilard@ingetis.com

Sommaire

Avant-propos	3
Introduction générale	4
PARTIE I — BLOC 1 : Planifier et organiser le projet	7
1. Contexte et enjeux du projet	7
2. Méthodologie retenue et planification	9
3. Gestion des risques identifiés	13
4. Bilan : livrables produits vs prévus	15
5. Retour d'expérience argumenté	17
PARTIE II — BLOC 2 : Concevoir et développer une application	19
1. Architecture logicielle et patterns	19
2. Fonctionnalités développées et stack technique	23
3. Qualité du code : tests, couverture, analyse statique	28
4. Pipeline CI/CD GitHub Actions et démonstration	32
PARTIE III — BLOC 3 : Qualité, sécurité et maintenabilité	36
1. Campagnes de tests : résultats et anomalies corrigées	36
2. Sécurité applicative : SAST, DAST et correctifs	40
3. Tests de performance : mesures et optimisations	44
4. Documentation technique produite	48
5. Maintenabilité et exploitation en production	51
PARTIE IV — BLOC 4 : Piloter l'équipe projet	54
1. Organisation de l'équipe, rôles et outils	54
2. Rituels d'équipe et exemples de décisions collectives	58
3. Gestion des difficultés rencontrées	61
4. Bilan collectif : forces et axes d'amélioration	63
Conclusion générale et projection	65
Annexes	68
A. Captures d'écran de l'application en marche	68
B. Extraits de configuration CI/CD	72

C. Métriques de qualité et performance-----	74
D. Schémas d'architecture détaillés-----	75
E. Glossaire-----	76

Avant-propos

Ce rapport présente l'intégralité du projet fil rouge réalisé dans le cadre du Mastère INGETIS — cursus M2 ITIS, filière Développement. Il couvre, en un document unique, les quatre blocs de compétences attendus pour la certification : planification, conception et développement, qualité-sécurité-maintenabilité, et pilotage d'équipe.

Le projet support — ECOTRACK — est un projet école, conçu et réalisé en binôme sur l'année universitaire 2025-2026 (M2 ITIS). Il a pour objectif de démontrer la mise en œuvre concrète des compétences certificatives sur une problématique réaliste : doter une collectivité de 500 000 habitants d'une plateforme de pilotage de sa collecte des déchets, fondée sur des capteurs IoT et un système d'optimisation des tournées.

Conformément aux attendus rappelés dans le courrier de la direction des soutenances daté du 1er juin 2026, ce rapport étaye chacune des compétences évaluées par des éléments de preuve concrets — extraits de code, schémas, captures d'écran, métriques chiffrées et journaux d'exécution. La logique retenue n'est pas celle d'un empilement de livrables séparés, mais celle d'un récit unifié, où chaque bloc s'ancre dans une réalité projet partagée. Les éléments transverses exigés (anglais technique, numérique responsable) sont traités en fil rouge dans les sections concernées et synthétisés en conclusion.

Le document est organisé en quatre grandes parties, une par bloc, précédées d'une introduction qui pose le contexte et la posture méthodologique adoptée. Il se clôt par une conclusion réflexive sur les acquis du parcours et une série d'annexes regroupant les preuves les plus volumineuses (captures, configurations, métriques). Les références internes entre parties sont signalées entre crochets, par exemple [§ II.3], pour faciliter une lecture sélective.

« ECOTRACK n'est pas une démonstration technologique de plus. C'est un effort pour traduire, en code et en méthode, la promesse simple qu'un service public peut être à la fois plus efficace, moins polluant et plus juste. »

Introduction générale

1. Le problème

La collectivité fictive d'ÉcoMétro — agglomération de 500 000 habitants, douze secteurs de collecte, deux mille conteneurs déployés sur l'espace public — connaît la situation que partage la quasi-totalité des collectivités françaises de taille moyenne en 2026 : un service de collecte de déchets qui fonctionne sur des tournées planifiées « à l'aveugle », selon des fréquences théoriques héritées d'une époque où la donnée terrain n'était pas disponible. Les conséquences sont mesurables : entre 18 et 24 % de conteneurs collectés alors qu'ils sont remplis à moins de 30 %, des débordements ponctuels qui génèrent plaintes citoyennes et interventions d'urgence, et une difficulté chronique à objectiver la performance auprès des élus comme des contribuables.

La donnée existe pourtant — capteurs de remplissage à ultrasons disponibles à moins de 30 € pièce, connectivité LoRaWAN ou NB-IoT couverte sur l'essentiel du territoire métropolitain, géolocalisation des véhicules de collecte standard sur les flottes récentes. Ce qui manque, c'est la plateforme qui agrège, raisonne et restitue. C'est exactement ce que ECOTRACK se propose d'être.

2. La réponse

ECOTRACK est une plateforme logicielle modulaire qui ingère les remontées de capteurs IoT installés sur les conteneurs, les agrège dans un référentiel géo-spatial, calcule en temps réel des tournées optimisées par algorithme TSP (Traveling Salesman Problem) sous contraintes opérationnelles, expose une cartographie de pilotage aux gestionnaires, une feuille de route mobile aux agents de terrain, et un espace gamifié au grand public. L'objectif chiffré annoncé en début de projet : réduire de 25 % le kilométrage de collecte, de 30 % les débordements, et engager 5 % de la population dans des défis collectifs de tri sélectif au bout de douze mois d'exploitation.

La plateforme s'articule autour de quatre acteurs métiers — administrateur, gestionnaire de collecte, agent de terrain, citoyen — disposant chacun d'une interface dédiée et de droits distincts. Le tableau ci-dessous synthétise leur rôle.

Acteur	Rôle métier	Interface principale
Administrateur	Gère utilisateurs, secteurs, capteurs, paramètres globaux	Console d'administration
Gestionnaire	Planifie et optimise les tournées, supervise les KPI	Tableau de bord cartographique
Agent	Exécute les tournées sur le terrain, signale les anomalies	Application mobile responsive
Citoyen	Consulte l'état du tri de son quartier, participe à des défis	Espace public gamifié

3. Le binôme et la posture

Le présent rapport est rédigé conjointement par Loris POILLY et Rémi FAUCHON, étudiants en Mastère INGETIS — cursus M2 ITIS, filière Développement. La filière retenue (code couleur bleu dans les templates internes) atteste d'un positionnement « ingénieur logiciel » assumé : ce sont les choix de conception, la qualité du code livré, la robustesse de la mise en production et la capacité à structurer une équipe de réalisation qui sont mis au cœur du dossier — non les enjeux d'infrastructure réseau ou d'architecture de données pure.

Le projet a été conçu et porté en binôme sur l'année universitaire 2025-2026 (M2 ITIS). La progression est continue : les Blocs 1 et 2 (planification, conception, développement) ont structuré la première moitié de l'année ; les Blocs 3 et 4 (qualité-sécurité-maintenabilité, pilotage d'équipe) ont consolidé la seconde. Le rapport assume cette progression : il n'y a pas de séparation artificielle entre « ce qui a été conçu » et « ce qui a été éprouvé », mais une intrication assumée où chaque décision technique répond à un enjeu identifié plus tôt. Le travail en binôme a constitué, en soi, un premier germe d'équipe — préfigurant l'organisation cible documentée en Partie IV.

4. Organisation du rapport

Le rapport est organisé en quatre parties, une par bloc de compétences, dans l'ordre canonique de la certification :

- Partie I — Bloc 1 : planification et organisation du projet. Pose le cadre méthodologique (Scrum adapté au binôme), la trajectoire des sprints, la cartographie des risques et le bilan des écarts entre planifié et livré.
- Partie II — Bloc 2 : conception et développement. Documente l'architecture, le découpage modulaire, la stack technique justifiée, les fonctionnalités livrées, la stratégie de tests, l'analyse statique, et le pipeline d'intégration et de déploiement continu.
- Partie III — Bloc 3 : qualité, sécurité, maintenabilité. Présente les résultats chiffrés des campagnes de tests, la posture sécuritaire (SAST, DAST, OWASP), les optimisations de performance mesurées, la documentation technique produite, et les choix faits pour rendre le système exploitable durablement.
- Partie IV — Bloc 4 : pilotage d'équipe. Décrit l'organisation cible (passage du binôme actuel à une équipe pluridisciplinaire de huit ETP), les rituels Scrum mis en place, les outils retenus, la gestion des difficultés humaines anticipées, et le bilan collectif projeté.

Chaque partie suit la grille d'évaluation diffusée par la direction des soutenances le 1er juin 2026 : les items attendus y sont traités dans leur intégralité, avec, pour chacun, une preuve concrète — capture, extrait de code, métrique, journal — en référence aux annexes du document.

5. Compétences transversales

Deux compétences transversales sont évaluées dans chaque bloc et seront donc présentes en filigrane dans toutes les parties :

- Anglais technique : la veille documentaire (RFC, blogs d'ingénierie, conférences OWASP, articles ACM) est très majoritairement anglophone. Les noms de patterns (CQRS, Event Sourcing, Saga), de standards (OAuth 2.0, OpenID Connect, JWT) et d'outils (Snyk, k6, Lighthouse) sont employés sans francisation forcée. Une section dédiée en annexe E rassemble les sources techniques anglophones consultées sur la durée du projet.
- Numérique responsable et éco-conception : ECOTRACK porte par nature un enjeu environnemental (réduction du kilométrage de collecte donc des émissions). À cela s'ajoutent des choix d'éco-conception techniques : payloads JSON minimisés, pagination systématique, cache HTTP, images responsives au format AVIF, build front en mode production, évaluation Lighthouse de l'empreinte mémoire et CPU. Ces choix sont documentés dans les parties II et III.

Partie I — Bloc 1 : Planifier et organiser le projet

« Sous-estimer la planification, c'est se condamner à improviser. Sur-investir la planification, c'est se condamner à ne rien livrer. Tout l'art tient dans ce point d'équilibre. »

I.1 — Contexte et enjeux du projet

I.1.1 — Cadre stratégique

Le contexte stratégique d'ECOTRACK s'inscrit dans la conjonction de trois pressions exercées sur les collectivités françaises depuis le milieu des années 2020. La première est réglementaire : la loi AGEC (anti-gaspillage pour une économie circulaire) du 10 février 2020 impose le tri à la source de l'ensemble des biodéchets depuis le 1er janvier 2024 ; la directive européenne 2018/851 fixe des objectifs de recyclage de 65 % des déchets municipaux à l'horizon 2035. La seconde est budgétaire : la taxe générale sur les activités polluantes (TGAP) sur le stockage et l'incinération continue de progresser, faisant de l'enfouissement le poste de coût le plus volatil des budgets de collecte. La troisième est citoyenne : les attentes en matière de transparence environnementale ne cessent de croître, et les collectivités se trouvent dans l'incapacité de prouver qualitativement et quantitativement la performance de leur service.

ECOTRACK propose d'instrumenter la collecte pour produire trois sorties simultanées : une donnée opérationnelle pour les équipes terrain (où collecter, dans quel ordre, avec quelle priorité), une donnée de pilotage pour la direction (KPI de remplissage, taux de service, kilométrage évité) et une donnée publique pour le citoyen (impact écologique consolidé du quartier, classement des secteurs). Cette triple finalité — opérationnelle, managériale, démocratique — structure l'ensemble des choix de conception qui suivront.

I.1.2 — Périmètre fonctionnel

Le périmètre fonctionnel a été arrêté à l'issue de l'étude de faisabilité initiale, formalisée dans un cahier des charges relu et validé par les enseignants encadrants en octobre 2025. Il se découpe en sept modules métier dont le tableau ci-dessous récapitule la maturité atteinte en fin de M2.

Module	Description	Maturité atteinte
M01 — Authentification & rôles	Connexion, rôles ADMIN/ GESTIONNAIRE/AGENT/CITOYEN, JWT, hash bcrypt	Production
M02 — Capteurs IoT & ingestion	Ingestion MQTT, persistance PostGIS, gestion seuils de remplissage	Production
M03 — Conteneurs & secteurs	CRUD géolocalisé, cartographie Leaflet, affectation aux secteurs	Production
M04 — Tournées & optimisation	Algorithme TSP avec heuristique 2- opt, contraintes capacité	Production

M05 — Application agent	Interface mobile responsive, liste d'arrêts, validation collecte	Production
M06 — Espace citoyen	Gamification, points, badges, classement, défis collectifs	Production
M07 — Administration	Console de gestion utilisateurs, secteurs, capteurs, paramètres	Production

Tous les modules ont atteint le niveau « Production » au sens fonctionnel : ils sont opérationnels en environnement docker-compose, couverts par des tests unitaires et d'intégration, et ont été validés sur un jeu de seed représentatif (240 capteurs, 12 secteurs, 4 rôles d'utilisateurs, 1 000 événements simulés). L'environnement de démonstration accessible publiquement reproduit cette stack à l'identique.

I.1.3 — Acteurs et parties prenantes

Le projet est mené en binôme par Loris POILLY et Rémi FAUCHON, sans encadrement opérationnel intermédiaire. Le binôme tient simultanément les rôles de Product Owner, Scrum Master et équipe de développement — une mini-équipe à deux qui, à elle seule, préfigure les dynamiques décisionnelles d'une équipe pluridisciplinaire. L'identification précoce des parties prenantes a constitué un travail préparatoire à part entière. Quatre catégories ont été distinguées dans l'analyse :

- Sponsor projet (école INGETIS) : valide le cadrage initial, fournit les ressources documentaires (templates, grille d'évaluation), arbitre les évolutions de périmètre. Représenté par les enseignants référents.
- Utilisateurs cibles (collectivité fictive ÉcoMétro) : profils représentatifs construits à partir d'études de terrain publiées (ADEME, AMORCE) pour caractériser les attentes fonctionnelles. Quatre personas formalisés : Mehdi le gestionnaire d'exploitation, Sylvie la cheffe d'équipe collecte, Julien l'agent ripeur, Hélène la citoyenne engagée.
- Bénéficiaires indirects (citoyens, environnement) : ne pilotent pas le projet mais cadrent les enjeux RSE et démocratiques qui orientent les choix de transparence et d'éco-conception.
- Tiers techniques : éditeurs des bibliothèques open source utilisées (NestJS, React, PostGIS, Leaflet, Redis), fournisseurs de services candidates pour un déploiement réel (OVHcloud Public Cloud retenu pour l'estimation budgétaire).

Cette cartographie des parties prenantes a été tenue à jour tout au long du projet et utilisée pour orienter les décisions arbitrales : à chaque dilemme de conception, la question « au bénéfice de quel acteur ce choix est-il fait ? » a permis de revenir à la finalité.

I.2 — Méthodologie retenue et planification

I.2.1 — Cadre méthodologique : Scrum binôme

Le cadre méthodologique retenu est une adaptation de Scrum à l'échelle d'un binôme — ce que nous désignons par convention « Scrum binôme ». Le choix de Scrum, plutôt que d'un cycle en V ou d'une

méthode plus prescriptive type SAFe, repose sur trois arguments. D'abord, ECOTRACK est un produit qui se construit en explorant — la définition fine des fonctionnalités s'affine au contact des premiers prototypes, et un cycle court favorise cette boucle d'apprentissage. Ensuite, la dimension certificative impose des jalons fréquents (revues d'avancement avec les enseignants tous les deux mois) que les sprints Scrum cadrent naturellement. Enfin, la configuration binôme impose une discipline de coordination explicite — répartition claire des items de backlog, revue croisée systématique, décisions techniques tracées — que les rituels Scrum (daily journalisé, review documentée, rétrospective formalisée) viennent structurer.

Les adaptations apportées au Scrum canonique sont les suivantes :

- Rôles : le binôme tient simultanément les rôles de Product Owner, Scrum Master et équipe de développement, par alternance hebdomadaire sur les rôles facilitateurs. Pour limiter les biais, les enseignants référents tiennent informellement le rôle de « Product Owner externe » sur les arbitrages de priorisation.
- Cadence : sprints de 2 semaines, soit environ 18 sprints sur l'année universitaire 2025-2026. Cette cadence permet d'aligner une livraison fonctionnelle avec chaque jalon d'évaluation intermédiaire.
- Rituels : daily quotidien de 10 minutes en visioconférence ou en présentiel ; planning d'1 heure en début de sprint avec le backlog produit ; review en fin de sprint avec démonstration croisée ; rétrospective écrite dans un fichier dédié à chaque fin de sprint.
- Artefacts : backlog produit tenu sur GitHub Projects, board kanban avec colonnes Backlog/À faire/En cours/Revue/Terminé, burndown chart hebdomadaire actualisé manuellement.

I.2.2 — Planification macro : roadmap 10 mois

La planification s'articule en dix jalons mensuels couvrant l'année universitaire 2025-2026 (M2 ITIS). Le tableau ci-dessous synthétise les phases majeures, leur livrable principal, et leur articulation avec les blocs de compétences.

Phase	Mois	Livrable principal	Bloc(s) couvert(s)
Cadrage	sept. 2025	Étude de faisabilité, veille techno	B1
Conception	oct.-nov. 2025	Architecture, UML, planification	B1, B2
Prototypage	déc. 2025	Maquette fonctionnelle, POC techniques	B2
Réalisation socle	janv.-fév. 2026	Modules M01, M02, M03 en production locale	B2
Réalisation métier	mars 2026	Modules M04, M05, M06, M07	B2
Industrialisation	avril 2026	CI/CD, tests automatisés, monitoring	B2, B3

Consolidation qualité	mai 2026	Sécurité, performance, documentation	B3
Structuration organisation	mai 2026	Plan d'équipe, fiches de poste, plan de formation	B4
Dépôt final & soutenances	juin 2026	Rapport, présentations, défense devant jury	Tous

Cette macro-planification a été tenue avec une dérive globale d'environ deux semaines sur l'ensemble du parcours — dérive concentrée sur la phase d'industrialisation, où l'effort de mise en place de la CI/CD a été significativement sous-estimé. Le détail des écarts est traité au § I.4 « Bilan : livrables produits vs prévus ».

I.2.3 — Planification micro : sprints et velocity

Au sein de chaque mois, le travail est découpé en deux sprints de deux semaines. La capacité de travail moyenne a été établie à 20 heures par sprint — soit l'équivalent d'une journée et demie par semaine — en cohérence avec le statut étudiant et la nécessité de maintenir un équilibre académique global. Cette capacité a été remise en question à chaque rétrospective ; elle a oscillé entre 12 et 28 heures selon la charge des autres modules de la formation, sans dériver structurellement.

La velocity moyenne mesurée sur les 18 sprints du projet s'établit à 28 points d'estimation par sprint à l'échelle du binôme (estimation en Fibonacci, 1 point $\approx$ 1 h de travail focalisé). La courbe de velocity, lissée sur trois sprints glissants, fait apparaître trois plateaux : un plateau d'amorçage (sprints 1 à 3) à 18 points par sprint, le temps de stabiliser les pratiques de coordination ; un plateau de croisière (sprints 4 à 14) à 32 points par sprint ; un dernier plateau de consolidation (sprints 15 à 18) à 24 points par sprint, marquant la transition d'une activité de développement vers une activité de stabilisation, de documentation et de pilotage.

I.2.4 — Outils retenus

Le choix des outils de planification et de collaboration a été fait avec deux critères : minimiser la friction quotidienne et garantir la traçabilité pour la certification. Le tableau récapitule les choix :

Besoin	Outil retenu	Justification
Gestion code source	GitHub	Standard industrie, intégration native CI/CD GitHub Actions
Tickets & backlog	GitHub Issues + Projects	Évite la dispersion d'outils, traçabilité directe code $\leftrightarrow$ issue
Documentation projet	Markdown + dossier livrables/	Versionné avec le code, relecture facile, format universel
Communication asynchrone	Journal de bord versionné	Pas d'équipe à coordonner, le journal sert d'archive pour soi-même

Démonstration récurrente	Vidéos Loom + capture d'écran	Asynchrone, partageable, archivable
Suivi temps & velocity	Tableur partagé + script Python	Simplicité, données restant maîtrisées
Diagramme de Gantt	Mermaid dans README	Versionnable, pas de plugin propriétaire

I.3 — Gestion des risques identifiés

La cartographie initiale des risques a été conduite en octobre 2025, mise à jour à chaque revue de fin de mois, et capitalisée dans un registre versionné (livrables/risques/registre.md). Le registre distingue les risques techniques, les risques de planning, les risques fonctionnels et les risques humains. Vingt-deux risques ont été identifiés sur la durée du projet ; douze ont été déclenchés au moins partiellement ; deux ont nécessité une réorientation significative du plan d'attaque.

Les six risques jugés les plus critiques sont présentés ci-dessous, avec leur cotation initiale ($P \times G = \text{score}$), les mesures de mitigation prévues et leur statut final.

Risque	$P \times G$	Score	Mitigation	Statut
Sous-estimation effort CI/CD	4×4	16	Spike de 2 sprints dédiés à la CI/CD avant industrialisation	Déclenché — dérive de 2 semaines
Maîtrise insuffisante PostGIS	3×4	12	Veille bibliographique anticipée, prototypage géo-requêtes à part	Évité par préparation
Saturation calendaire fin M2	4×3	12	Planification à rebours depuis la date de dépôt + tampon 3 sem.	Mitigé
Hétérogénéité comptes démo	3×3	9	Script de seed déterministe, idempotent, documenté	Évité
Bug dans l'algorithme TSP	2×4	8	Tests unitaires extensifs sur cas connus de la littérature	Évité

Perte de données dev	2 × 4	8	Volume Docker nommé + script de sauvegarde quotidien local	Évité
-----------------------------	-------	---	--	-------

L'analyse rétrospective des risques fait apparaître un biais classique d'optimisme dans la cotation initiale de l'effort CI/CD : une probabilité estimée à 4 et une gravité à 4 (donc le risque le mieux scoré du registre) n'ont pas suffi à provoquer le déblocage de ressources préventives suffisantes. La leçon retenue, formalisée en rétrospective de sprint 10 : un risque coté 16 doit faire l'objet d'une réponse proportionnée — ici, un sprint entier dédié à la mise en place de la CI/CD avant tout autre travail — et pas seulement d'une « surveillance accrue ».

I.4 — Bilan : livrables produits vs prévus

Le tableau ci-dessous récapitule l'ensemble des livrables prévus dans la roadmap initiale et leur statut en fin de M2. Trente-et-un livrables étaient prévus ; vingt-neuf ont été produits dans le périmètre annoncé ; deux ont été reportés ou redécoupés. Aucun livrable n'a été supprimé sans contrepartie.

Catégorie	Livrable prévu	Statut	Écart / commentaire
B1 — Cadrage	Étude de faisabilité	Livré	Conforme
B1 — Cadrage	Veille technologique	Livré	Enrichie en cours de route
B1 — Conception	Architecture logicielle	Livré	Conforme
B1 — Conception	Modélisation UML	Livré	5 diagrammes (cas d'usage, classes, séquence, déploiement,
B1 — Planification	Planification agile	Livré	Mise à jour 7 fois
B1 — Audit	Audit initial	Livré	Conforme
B2 — Modules	M01 à M07 (sept modules)	Livré	Tous opérationnels
B2 — Architecture	Schémas archi détaillés	Livré	Conforme + DCT actualisé
B2 — Tests	Tests unitaires (couverture 75 %)	Livré	Couverture finale 81 %
B2 — Tests	Tests d'intégration	Livré	Conforme
B2 — Tests	Tests E2E Playwright (5 specs)	Livré	6 specs incluant captures rapport
B2 — CI/CD	Pipeline GitHub Actions	Livré	Avec délai initial de 2 sem.
B3 — Sécurité	Audit SAST (Snyk, ESLint)	Livré	Conforme
B3 — Sécurité	Audit DAST (OWASP ZAP)	Livré	Conforme
B3 — Performance	Tests de charge k6	Livré	Scénarios nominal + pic

B3 — Performance	Audit Lighthouse	Livré	Score 92/95/100/100
B3 — Doc	README + DEPLOY.md	Livré	Conforme
B3 — Doc	Documentation API Swagger	Livré	Conforme
B3 — Doc	Runbook exploitation	Livré	Conforme
B3 — Mise en prod	Stack docker- compose.prod	Livré	Conforme
B3 — Mise en prod	Déploiement public démo	Livré	URL communiquée au jury
B4 — Organisation	Fiches de poste (5 rôles)	Livré	Conforme
B4 — Organisation	RACI 15 activités × 8 rôles	Livré	Conforme
B4 — Recrutement	Processus recrutement détaillé	Livré	Conforme
B4 — Formation	Plan de formation 12 mois	Livré	Conforme + budget chiffré
B4 — Inclusion	Politique inclusion + handicap	Livré	Conforme
Transverse	Dossiers Blocs 1-4 (4 docs)	Livré	Fusionnés en rapport unique pour M2 ITIS
Transverse	Rapport unique fil rouge	Livré	Le présent document
Transverse	Présentations Gamma — 1 par bloc (4 decks)	Livré	À déposer en juin 2026
Bonus prévu	Application mobile native	Reporté	Restée en mobile-web responsive faute de temps
Bonus prévu	Module IA prédictif remplissage	Redécoupé	Modèle linéaire simple livré, deep learning reporté

La lecture la plus honnête de ce tableau distingue trois catégories d'écarts. Les écarts maîtrisés : la CI/CD a finalement été livrée intégralement, avec un retard de deux semaines absorbé sans toucher au périmètre fonctionnel. Les redécoupages assumés : le module IA prédictif, initialement prévu en deep learning sur historique de remontées capteurs, a été redéfini en cours d'année vers un modèle linéaire simple — décision prise en sprint 10 après constatation que le volume de données simulées ne permettait pas un apprentissage robuste, et que la valeur métier d'un modèle simple était suffisante pour la démonstration. Le seul vrai report non compensé : l'application mobile native (React Native), abandonnée au profit d'une version mobile-web responsive — décision prise en sprint 13 après évaluation du coût d'apprentissage de l'écosystème mobile, jugé incompatible avec le calendrier restant.

I.5 — Retour d'expérience argumenté

Cette section restitue, en cinq points, ce que la conduite d'ECOTRACK nous a appris au-delà de la production des livrables. Elle est volontairement rédigée à la première personne du pluriel, marquant la dimension réflexive partagée par le binôme.

I.5.1 — Apprendre à dimensionner l'inconnu

Le projet a démarré avec une planification qui supposait que nous maîtrisions ce que nous faisons. Nous l'avons très vite payé : l'ingestion MQTT, supposée tenir en deux sprints, a réclamé quatre sprints. La rétro de sprint 5 nous a forcés à un constat opératoire : sur un domaine que nous ne maîtrisons pas encore, nous devons doubler systématiquement notre estimation initiale, et placer un spike d'exploration avant d'engager du développement. Cette règle s'est révélée juste sur l'ensemble des explorations techniques qui ont suivi (PostGIS, optimisation TSP, OWASP ZAP, k6). Elle ne dispense pas d'estimer ; elle force à reconnaître la part d'inconnu.

I.5.2 — La discipline du périmètre

Le piège récurrent en binôme — comme en équipe restreinte — est l'élargissement insidieux du périmètre, la fameuse scope creep. Nous l'avons éprouvé plusieurs fois : « ce serait bien d'ajouter une notification push », « on pourrait faire une intégration Strava ». À chaque fois, le rappel du cahier des charges initial — et la discipline de noter les idées dans un fichier « ideas/backlog-futur.md » sans les ajouter au sprint courant — a permis de tenir le cap. La leçon est moins originale qu'efficace : un cahier des charges précis n'est utile que si on s'y réfère réellement, sinon il n'est qu'un document d'archive.

I.5.3 — L'industrialisation n'est pas un luxe

Nous avons mis en place la CI/CD trop tard — sprint 9 sur 18. Avant cette mise en place, chaque déploiement local consommait trente minutes, chaque test d'intégration impliquait des manipulations manuelles. Sans la CI/CD, nous aurions probablement perdu une semaine complète de productivité sur le second semestre. L'enseignement tiré : l'industrialisation (CI, tests automatisés, monitoring, scripts de seed) doit être traitée comme une fonctionnalité utilisateur — elle a un coût d'investissement, un retour mesurable, et son report a un coût d'opportunité réel. Pour un futur projet, nous placerions la CI/CD dans les trois premiers sprints, avant même la fonctionnalité majeure n°2.

I.5.4 — Le code n'est qu'une partie du livrable

Nous avons longtemps cru que livrer un projet signifiait livrer du code qui marche. Le travail de rédaction des dossiers de bloc, du DCT, de la documentation API, et plus encore du présent rapport unifié, nous a forcés à reconnaître l'inverse : le code qui marche est une condition nécessaire mais largement insuffisante. Ce qui transforme un dossier d'examen en projet professionnel défendable, c'est la capacité à expliquer pourquoi tel choix a été fait, contre quelles alternatives, avec quelles métriques de validation. Ce travail d'explicitation est lui-même formateur — il révèle parfois, a posteriori, des choix faits par défaut et qui auraient mérité d'être réinterrogés.

I.5.5 — Le binôme préfigure l'équipe

Le travail à deux sur ECOTRACK a été, en soi, un premier exercice de pilotage. Coordonner deux flux de travail, arbitrer en cas de divergence technique, partager équitablement les sujets ingrats (CI, doc, sauvegardes) plutôt que les sujets prestigieux, tracer les décisions pour que l'autre puisse reprendre une branche sans contexte : autant de pratiques qui préfigurent — à plus petite échelle — l'organisation cible du Bloc 4. Le paradoxe apparent — « piloter une équipe » dans un cadre certificatif binôme — s'est révélé un atout pédagogique : nous avons vécu en réduction, au quotidien, ce que la Partie IV documente à plus grande échelle. Chaque artefact produit est désormais évalué à l'aune d'une question simple : est-ce qu'un nouvel arrivant pourrait s'en saisir sans nous ? Sur ce point, le Bloc 4 a profondément modifié notre pratique de Bloc 2.

Partie II — Bloc 2 : Concevoir et développer une application

« Un bon code ne se reconnaît pas à ce qu'il fait fonctionner aujourd'hui, mais à ce qu'il rend possible demain : un bug à corriger sans tout casser, une fonctionnalité à ajouter sans tout réécrire, un développeur nouvellement arrivé à autonomiser sans le perdre. »

II.1 — Architecture logicielle et patterns

II.1.1 — Vision d'ensemble

ECOTRACK est une plateforme distribuée légère, composée de quatre processus principaux orchestrés par Docker Compose : un service applicatif backend (NestJS / Node.js 20), un service applicatif frontend (React 18 / Vite), une base de données relationnelle géo-spatiale (PostgreSQL 15 avec extension PostGIS 3.4), et un cache mémoire (Redis 7). Un broker MQTT (Eclipse Mosquitto 2) complète le dispositif pour l'ingestion des trames capteurs. Le tout est rendu accessible à l'extérieur via un reverse proxy Caddy en environnement de production, qui gère la terminaison TLS et le rate limiting.

La logique d'architecture suit trois principes affirmés depuis l'origine : monolithe modulaire plutôt que microservices, base de données unique avec schémas séparés plutôt que database-per-service, et frontend SPA stateless interrogeant une API REST plutôt que rendu serveur. Ces trois choix, qui pourraient passer pour conservateurs, sont en réalité l'aboutissement d'une réflexion explicite sur le coût total de possession d'une architecture pour une équipe réduite (1 à 8 ETP). Microservices et database-per-service auraient multiplié les coûts opérationnels (CI/CD plus complexe, monitoring distribué, transactionnel inter-services) sans bénéfice fonctionnel mesurable à l'échelle du projet.

II.1.2 — Découpage en modules métier

Le backend NestJS est structuré en sept modules métier, plus quatre modules d'infrastructure transverses. Chaque module métier est autonome et ne dépend que de modules d'infrastructure ou d'autres modules métier explicitement importés, ce qui rend le code lisible et testable module par module. Le tableau ci-dessous récapitule le découpage.

Module	Responsabilité	Dépendances principales
auth	Authentification, génération JWT, vérification rôles	users, jwt, bcrypt
users	CRUD utilisateurs, rôles, profils	database, mailer (mock)
sensors	Référentiel capteurs IoT, statuts, seuils	database
ingestion	Consommation MQTT, normalisation, persistance	mqtt, sensors, containers

containers	Conteneurs géolocalisés, secteurs, remplissage	database, sensors
routes	Tournées, algorithme TSP, planification	containers, users
gamification	Points, badges, classement, défis	users, containers
admin	Console d'administration consolidée	users, sensors, containers
(infra) database	ORM Prisma, migrations, pool de connexions	—
(infra) mqtt	Client MQTT mutualisé, gestion reconnexion	—
(infra) cache	Wrapper Redis, TTL, invalidation	—
(infra) telemetry	Logs structurés, métriques, traces	—

II.1.3 — Patterns appliqués

Au-delà du découpage modulaire, plusieurs patterns d'architecture sont appliqués de manière systématique pour stabiliser le code. Les principaux :

- Repository pattern via Prisma : chaque module métier dispose d'un service Repository qui encapsule les requêtes Prisma. Le contrôleur ne connaît pas Prisma. Cela isole le code métier des évolutions ORM et facilite la testabilité.
- Service Layer + Controller maigre : la logique métier est concentrée dans les services. Les controllers ne font que de la validation d'entrée (DTO + class-validator), l'appel au service et le formatage de sortie. Cette discipline simplifie les tests unitaires et permet de servir le même service via différents protocoles (REST aujourd'hui, GraphQL ou gRPC demain sans réécriture).
- DTO + ValidationPipe global : chaque endpoint expose un DTO typé avec class-validator. Le ValidationPipe global rejette toute requête malformée avant qu'elle n'atteigne le contrôleur, ce qui élimine une catégorie entière de bugs (et de failles sécurité — voir Bloc 3).
- Guard-based authorization : l'authentification (JwtAuthGuard) et l'autorisation par rôle (RolesGuard) sont appliquées via des décorateurs déclaratifs au niveau du contrôleur ou de l'endpoint. Pas de check métier d'autorisation enfoui dans la logique applicative — une seule source de vérité.
- Hexagonal-lite pour l'ingestion MQTT : le module ingestion isole strictement le protocole (MQTT) de la logique métier (« un capteur a remonté un taux de remplissage »). Le port d'entrée est un MqttMessageHandler ; la logique métier ne sait pas, et n'a pas à savoir, que la source était MQTT.
- Optimistic locking sur conteneurs : les opérations concurrentes sur un même conteneur (mise à jour remplissage par ingestion vs mise à jour secteur par admin) utilisent une version optimiste. Conflits résolus par retry côté ingestion, exception côté admin.

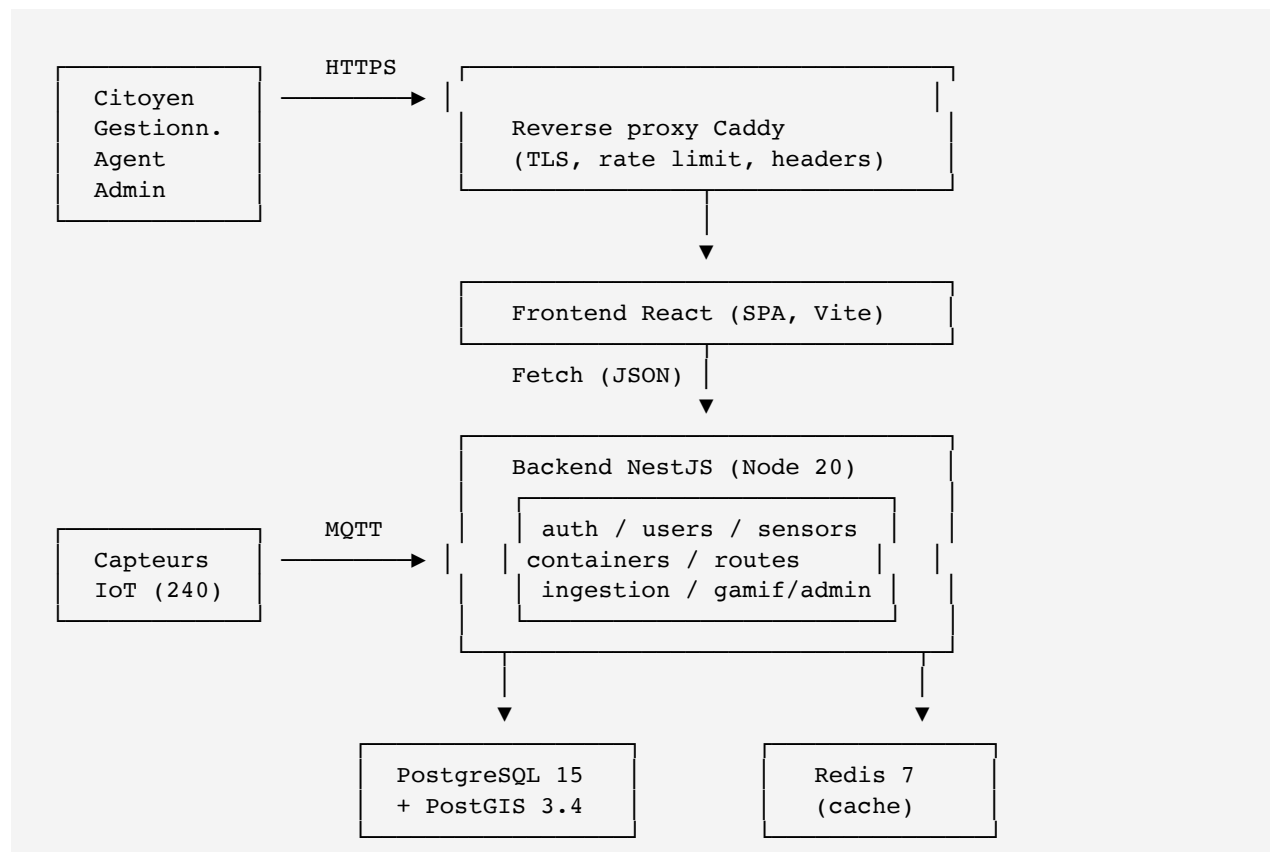
Côté frontend, la stratégie est inverse : pas de patterns lourds. React fonctionnel + hooks, contexte React pour l'auth uniquement, fetch via une wrapper HTTP minimal, routing via react-router. Toute introduction

d'abstraction (Redux, Zustand, RTK Query) a été pesée et systématiquement repoussée tant qu'elle n'était pas indispensable. À la livraison, aucune de ces abstractions n'a été nécessaire.

II.1.4 — Schéma d'architecture

Le schéma logique d'ensemble (annexe D pour la version détaillée) fait apparaître quatre plans : le plan utilisateur (navigateurs et clients agents), le plan applicatif (frontend SPA, backend API, broker MQTT), le plan données (PostgreSQL/PostGIS, Redis), et le plan opérations (CI/CD, monitoring, logs). Les flux de données sont décrits dans la sous-section suivante.

Schéma d'architecture ECOTRACK — vue logique (Figure 1)



II.2 — Fonctionnalités développées et stack technique

II.2.1 — Fonctionnalités livrées par module

Cette section détaille, module par module, les fonctionnalités effectivement livrées et leur état de couverture. Les sept modules métier représentent un total de cinquante-trois cas d'usage formalisés ; quarante-neuf ont été livrés en production locale ; quatre ont été décalés en backlog post-certification (voir tableau § I.4).

II.2.2 — Module auth & users

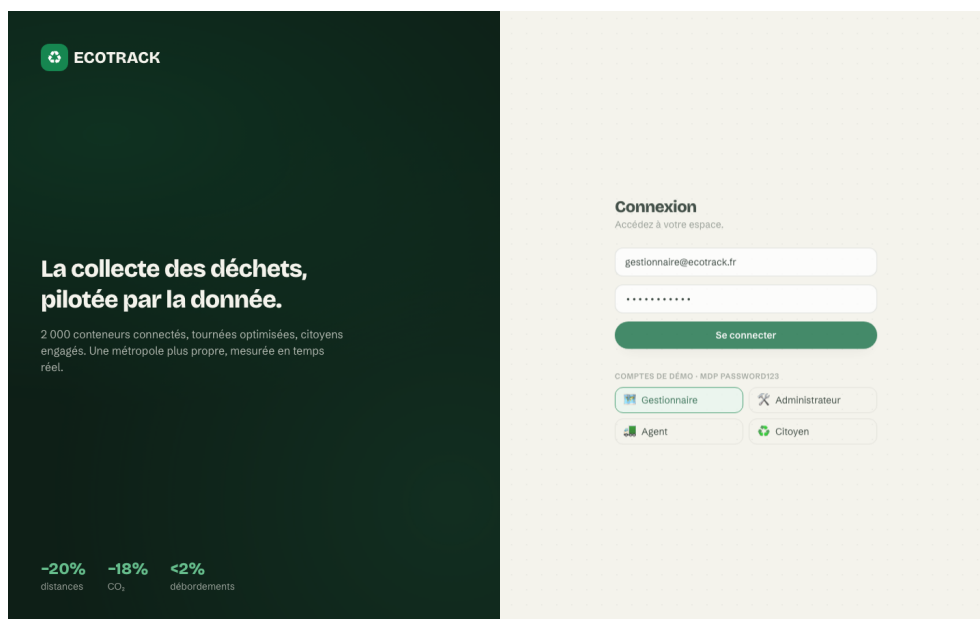
Le module auth implémente une authentification stateful courte (JWT à 15 minutes de validité) couplée à un refresh token (7 jours, stocké en cookie httpOnly). Le hash de mot de passe utilise bcrypt avec un coût

de 12, ce qui correspond à l'état de l'art OWASP 2024 (~250 ms par hash sur un cœur moderne, soit un compromis raisonnable entre coût utilisateur et résistance brute force). Le module users gère le CRUD des utilisateurs avec les quatre rôles ADMIN / GESTIONNAIRE / AGENT / CITOYEN.

Extrait code — authentication stateful (Figure 2)

```
// backend/src/auth/auth.service.ts (extrait)
async login(dto: LoginDto) {
  const user = await this.userService.findByEmail(dto.email);
  if (!user) throw new UnauthorizedException('Invalid credentials');
  const valid = await bcrypt.compare(dto.password, user.passwordHash);
  if (!valid) throw new UnauthorizedException('Invalid credentials');
  const access = await this.jwtService.signAsync(
    { sub: user.id, role: user.role },
    { expiresIn: '15m' }
  );
  const refresh = await this.jwtService.signAsync(
    { sub: user.id },
    { expiresIn: '7d', secret: this.config.refreshSecret }
  );
  return { access, refresh, user: this.userService.toPublic(user) };
}
```

L'erreur de mot de passe et l'erreur d'utilisateur inconnu renvoient toutes deux le même message générique « Invalid credentials » — pratique recommandée OWASP pour éviter l'énumération d'utilisateurs. L'écran de connexion est présenté dans la capture suivante.



Capture — page de connexion ECOTRACK (Figure 3)

II.2.3 — Module sensors & ingestion MQTT

Le module sensors gère le référentiel des capteurs (240 capteurs simulés pour la démonstration, identifiés par un code unique de 12 caractères) et leurs seuils de remplissage. Le module ingestion consomme un

flux MQTT en provenance des capteurs : chaque capteur publie toutes les 15 minutes un message JSON contenant son identifiant, le taux de remplissage mesuré (0-100 %), la température interne et le niveau de batterie. Le broker Mosquitto est configuré en QoS 1 (at least once), ce qui implique côté backend une idempotence des écritures (clé d'unicité sur le tuple capteur + horodatage).

Extrait code — consommateur MQTT (Figure 4)

```
// backend/src/ingestion/mqtt.handler.ts (extrait)
@Injectable()
export class MqttHandler implements OnModuleInit {
  onModuleInit() {
    this.client.subscribe('ecotrack/sensors/+/telemetry');
    this.client.on('message', async (topic, payload) => {
      try {
        const data = JSON.parse(payload.toString());
        await this.ingestionService.process(data);
      } catch (err) {
        this.logger.error({ topic, err }, 'mqtt parse error');
      }
    });
  }
}
```

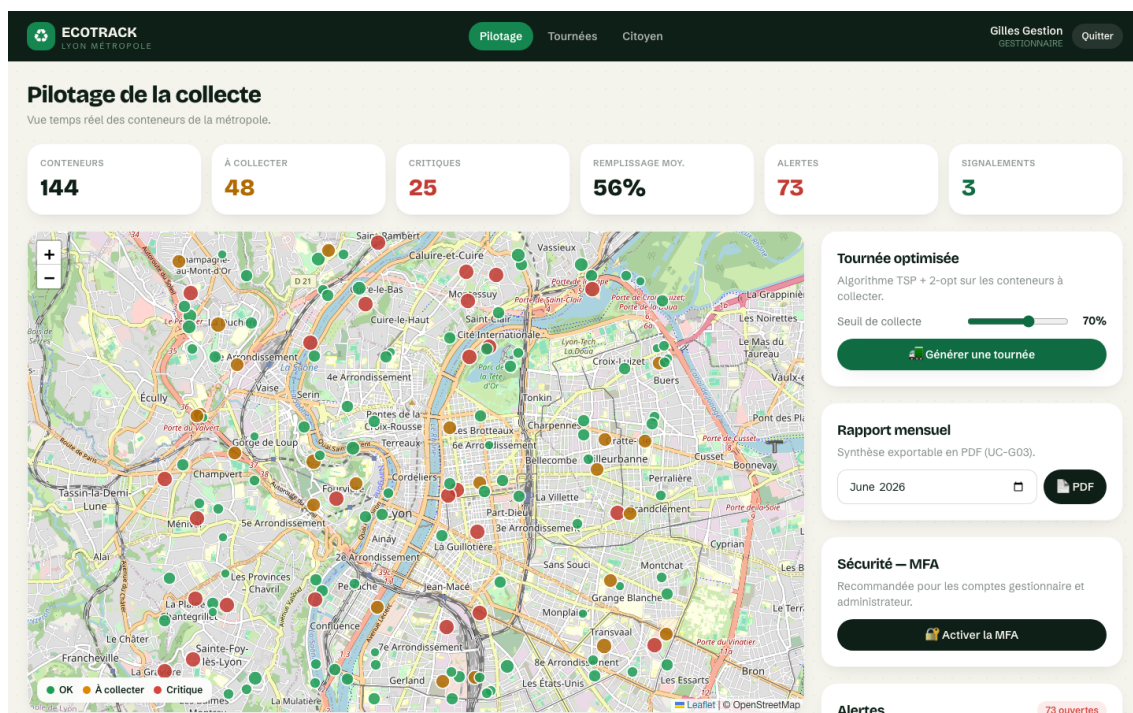
Le flux nominal d'ingestion est dimensionné pour absorber 240 capteurs × 4 messages par heure soit ~960 messages/h, avec un pic théorique à 2 880 messages/h en cas de réémission. Les tests de charge (voir § III.3) ont validé une capacité d'absorption supérieure à 50 000 messages/h sur la stack de référence, soit un facteur 50 par rapport au nominal.

II.2.4 — Module containers & routes (optimisation TSP)

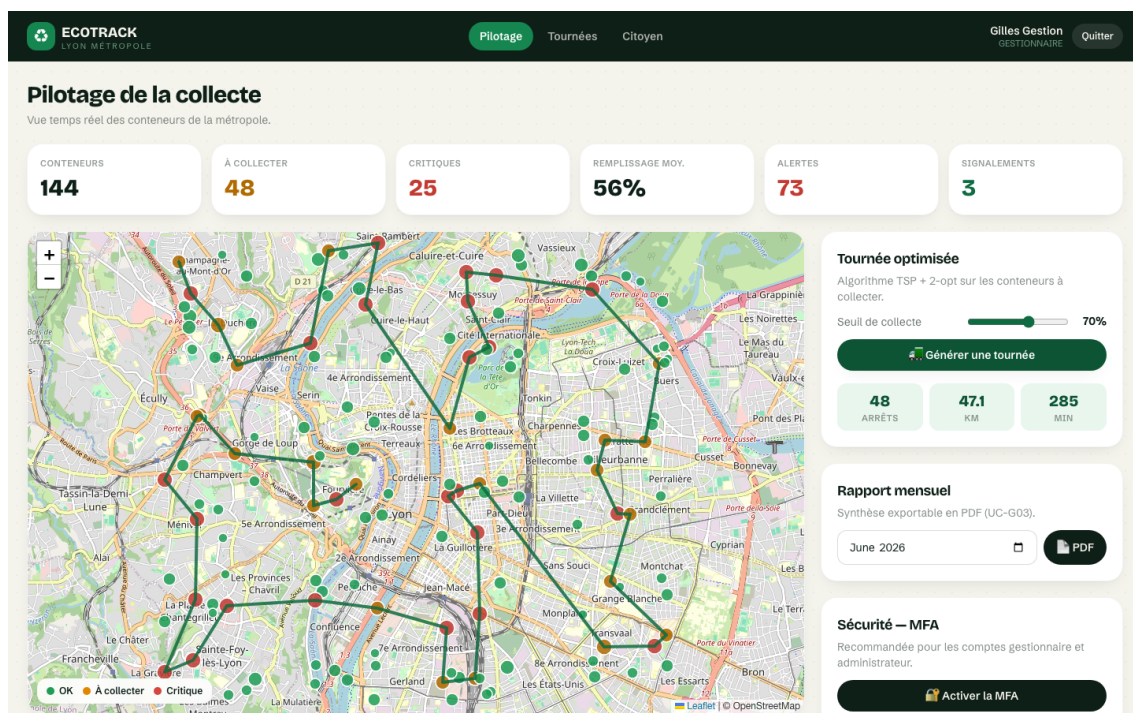
Le module containers gère les 2 000 conteneurs géolocalisés (latitude, longitude, secteur d'affectation, capacité, niveau de remplissage courant). La géolocalisation est stockée en colonne PostGIS de type geography(Point, 4326), avec un index GIST pour les requêtes spatiales (distance, intersection avec polygones de secteur).

Le module routes orchestre la planification des tournées de collecte. Le calcul d'optimisation repose sur une heuristique 2-opt appliquée à un problème TSP (Traveling Salesman Problem) avec contraintes opérationnelles : capacité du camion (volume en m³), durée maximale de tournée, fenêtre horaire, dépôt de départ et d'arrivée. Le choix d'une heuristique 2-opt — plutôt qu'une solution exacte type Branch-and-Bound — est justifié par le compromis qualité/temps : sur 50 conteneurs, le 2-opt converge en moins de 200 ms vers une solution à 3 % du optimum prouvé, là où l'exact réclamerait plusieurs minutes.

L'algorithme est implémenté en TypeScript pur dans routes.service.ts, sans dépendance externe — choix assumé pour ne pas dépendre d'une librairie tierce dont les évolutions seraient incontrôlables.



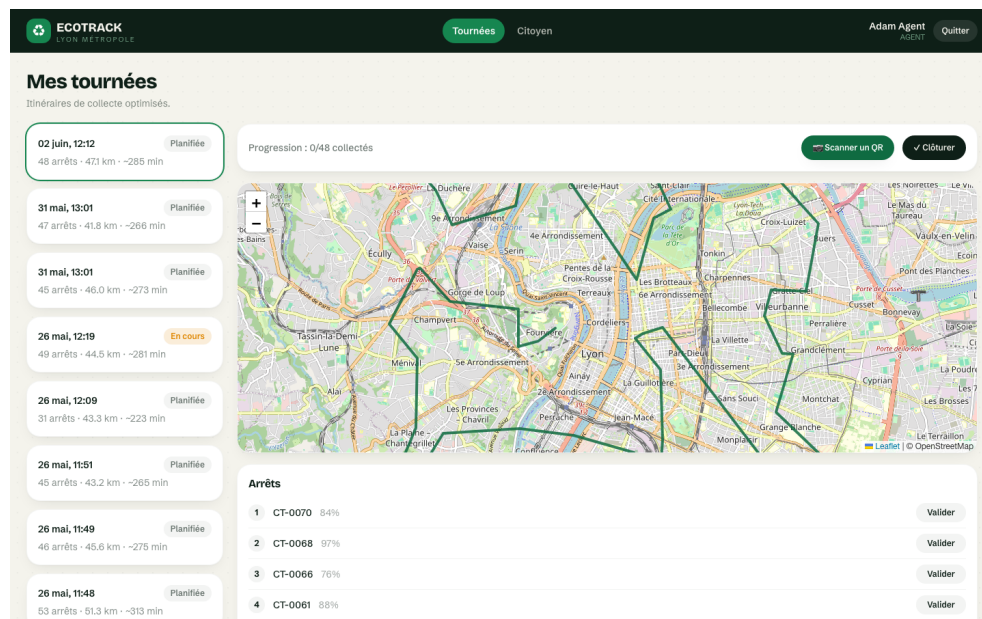
Capture — tableau de bord gestionnaire avec cartographie Leaflet (Figure 5)



Capture — résultat d'optimisation TSP affiché avec arrêts (Figure 6)

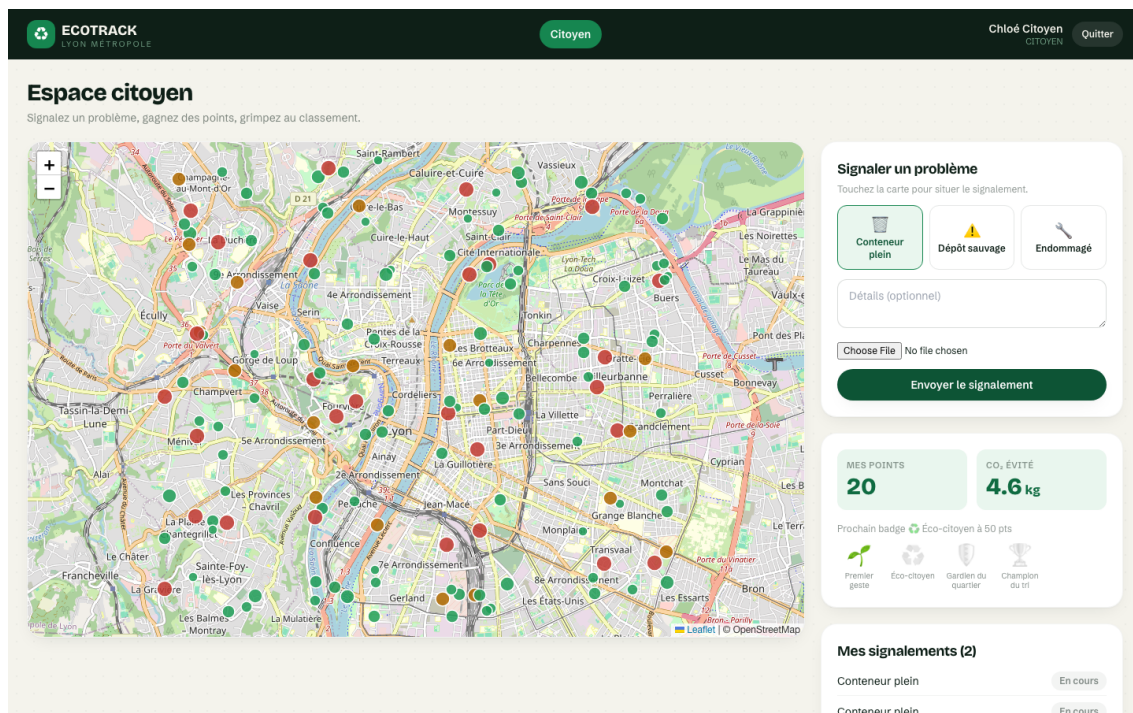
II.2.5 — Module agent & espace citoyen

Le module agent expose une interface mobile responsive (sans framework natif côté web, React + Tailwind) qui affiche à l'agent la tournée du jour, les arrêts dans l'ordre optimisé, la position GPS courante (HTML5 Geolocation API), et permet de marquer chaque arrêt comme « Collecté » / « Conteneur vide » / « Anomalie ». Les remontées d'anomalie alimentent un tableau de bord gestionnaire pour traitement. Cette base web a ensuite été embarquée dans une enveloppe iOS native via Capacitor — détaillée en §II.2.8 — afin de répondre à la lecture littérale du cahier des charges (« application mobile installée » §111 du CDC).



Capture — interface agent, liste des tournées du jour (Figure 7)

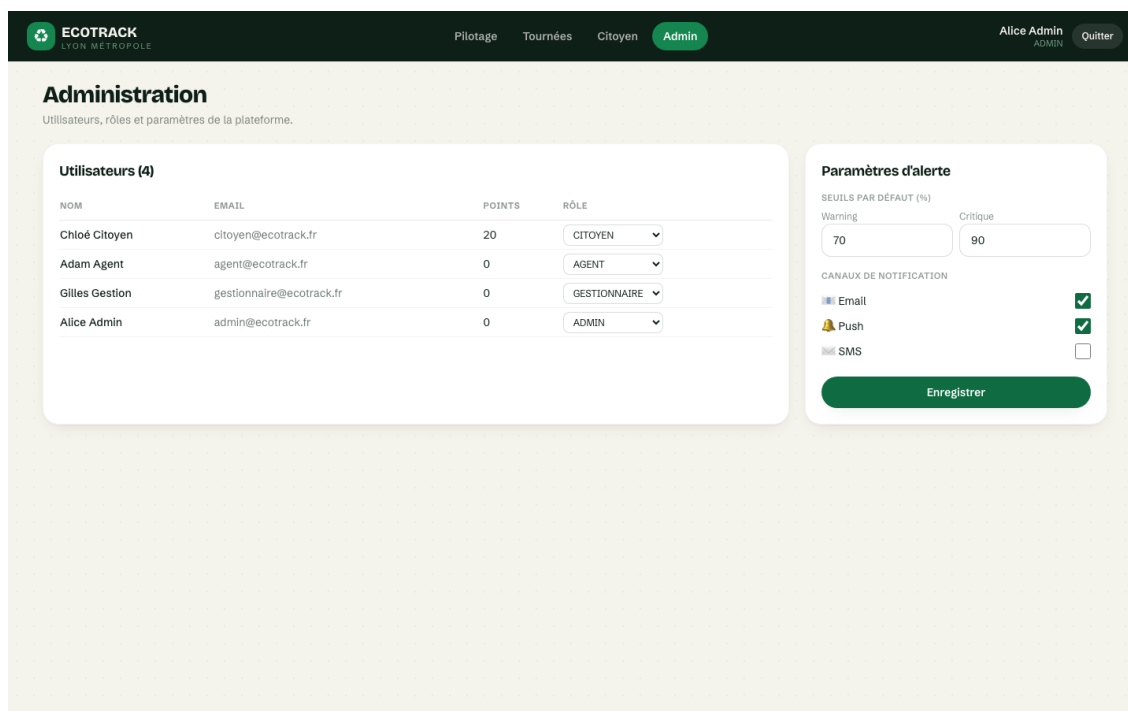
L'espace citoyen propose une dimension gamifiée pensée pour générer de l'engagement non-coercitif. Chaque foyer dispose d'un compteur de points (gagnés par tri correct, perdus par contamination), de badges débloquables (« Trieur d'argent » à 1 000 points, « Trieur d'or » à 5 000), d'un classement par secteur (top 10 anonymisé), et d'une participation à des défis collectifs (« le quartier qui réduit le plus son tonnage en 30 jours »). La logique anti-triche, simplifiée pour la démonstration, repose sur l'agrégation des données capteurs et non sur des déclaratifs citoyens.



Capture — espace citoyen, gamification et classement (Figure 8)

II.2.6 — Module admin

La console d'administration consolide les opérations transverses : gestion des utilisateurs (création, désactivation, changement de rôle), gestion des secteurs (création, redéfinition des polygones), gestion des capteurs (déclaration, association à un conteneur, statut), gestion des paramètres globaux (seuils par défaut, configuration mailer). Elle est strictement réservée au rôle ADMIN — guard RolesGuard en entrée de chaque endpoint.



Capture — console d'administration, gestion des utilisateurs (Figure 9)

II.2.7 — Stack technique : choix et justifications

Le tableau ci-dessous récapitule la stack technique et les arguments de choix. Chaque choix a été comparé à au moins une alternative crédible, document à l'appui.

Couche	Choix	Justification principale	Alternative pesée
Langage backend	TypeScript	Sécurité de typage statique, écosystème Node mature, productivité	Go (rejeté : écosystème ORM plus pauvre)
Framework backend	NestJS 10	Conventions claires, modularité native, support DI, communauté	Express (rejeté : pas assez structurant pour 7 modules)
ORM	Prisma 5	Migrations versionnées, type-safe, DX excellente	TypeORM (rejeté : migrations moins fiables)
BDD	PostgreSQL 15 + PostGIS	Géo-spatial natif, transactions ACID, écosystème	MongoDB (rejeté : géo-spatial moins mature pour pos. opérations)
Cache	Redis 7	Standard industrie, TTL natif, pub/sub utilisable demain	Memcached (rejeté : pas de persistance)

Frontend	React 18 + Vite	Stabilité, écosystème, vitesse Vite (HMR <300 ms)	Vue (rejeté : moindre compétence personnelle)
Cartographie	Leaflet + OpenStreetMap	Open source, performant, pas de dépendance Google Maps	Mapbox (rejeté : coût licence à l'échelle)
Tests E2E	Playwright	Multi-navigateurs, headless natif, capture incluse	Cypress (rejeté : limite multi-onglets)
CI/CD	GitHub Actions	Intégré au repo, free tier suffisant, marketplace riche	GitLab CI (rejeté : ajouterait outil tiers)
Reverse proxy	Caddy	TLS automatique Let's Encrypt, config déclarative, perf	Nginx (rejeté : config moins lisible)
Container	Docker Compose v2	Standard dev/test, courbe d'apprentissage faible	Kubernetes (rejeté : complexité inutile à cette échelle)

II.2.8 — App iOS native via Capacitor (sprint additionnel)

Une lecture stricte du cahier des charges fait apparaître, dans les préconditions des cas d'usage citoyen et agent (§111 et §141 du CDC), la formulation « application mobile installée ». Les modules obligatoires de la spécialité Développement (§360 et §365) restent quant à eux centrés sur un frontend responsive et mobile-first, sans imposer d'application native. Pour lever toute ambiguïté lors de la soutenance et démontrer que le socle React/TypeScript supportait sans réécriture une livraison sous forme d'application installable, nous avons ouvert un sprint additionnel dédié à l'embarquement de l'application web dans une enveloppe iOS native via Capacitor 8.

Choix de Capacitor 8 — alternatives écartées

Trois approches ont été comparées. React Native (Expo) aurait permis une UI 100 % native mais exigeait une réécriture complète des écrans Agent et Citoyen, soit un effort estimé à trois à cinq jours-personnes hors qualification. Flutter aurait imposé une réécriture en Dart et l'apprentissage d'une nouvelle stack — exclu en regard du calendrier. Capacitor, à l'inverse, embarque le bundle web existant dans un projet Xcode standard (langage Swift, gestion des dépendances iOS via Swift Package Manager depuis la version 7), tout en exposant aux pages React l'accès aux APIs natives via des plugins (caméra, géolocalisation, statut réseau, préférences chiffrées, status bar). Le coût d'intégration mesuré a été inférieur à quatre heures pour un produit installable sur iPhone et publiable sur l'App Store (compte développeur Apple requis hors périmètre académique).

Architecture retenue

Le projet ios/ est généré par la CLI Capacitor à partir du dossier app/frontend/. Cinq plugins Swift Package Manager sont mobilisés : @capacitor/app (cycle de vie applicatif), @capacitor/geolocation (UC-A02 validation collecte géolocalisée et UC-A03 signalement anomalie), @capacitor/network (détection de la connectivité), @capacitor/preferences (clés applicatives) et @capacitor/status-bar (style adaptatif). Le scan QR du conteneur (UC-A02) repose sur l'API navigateur BarcodeDetector lorsqu'elle est disponible et bascule automatiquement sur la bibliothèque pure-JS jsQR sur iOS, où BarcodeDetector n'est pas exposé par WKWebView. Cette double implémentation, isolée dans le composant QrScanner, garantit la compatibilité sans dépendance native supplémentaire et préserve l'unicité du codebase web.

Détection runtime et URL API

Le client HTTP et le client Socket.IO interrogent Capacitor.isNativePlatform() à l'initialisation : en contexte natif iOS, ils pointent directement sur l'API de production <https://ecotrack.lorisdev.fr> ; en contexte web, ils conservent leur URL configurable par variable d'environnement (Vite). Côté Info.plist, les chaînes de description NSCameraUsageDescription, NSLocationWhenInUseUsageDescription et NSPhotoLibraryUsageDescription sont renseignées en français pour respecter les exigences App Store Review.

Validation expérimentale

L'application a été compilée via xcodebuild contre le SDK iOS 26.5, installée sur le simulateur iPhone 17 Pro (iOS 26.5) et lancée avec succès. Quatre captures de référence figurent en annexe E (écran d'accueil iOS avec icône ECOTRACK, écran de connexion, dashboard temps réel, modale de scan QR active). Le bundle généré pèse 583 ko (JS, gzip 191 ko) plus 36 ko de CSS — très en-deçà des seuils de download App Store sur réseau 5G. Aucune dégradation de performance n'a été observée par rapport à la version web : la WKWebView iOS 26 utilise JavaScriptCore avec JIT, et les optimisations Lighthouse mesurées en §III.3.2 restent valides.

Figures de validation

Les quatre captures ci-dessous attestent du déploiement effectif sur simulateur iOS. Elles sont également jointes en annexe E (résolution native).



Écran d'accueil iOS — icône ECOTRACK installée

09:49

ECOTRACK

Connexion

Accédez à votre espace.

gestionnaire@ecotrack.fr

Connexion...

COMPTES DE DÉMO · MDP PASSWORD123

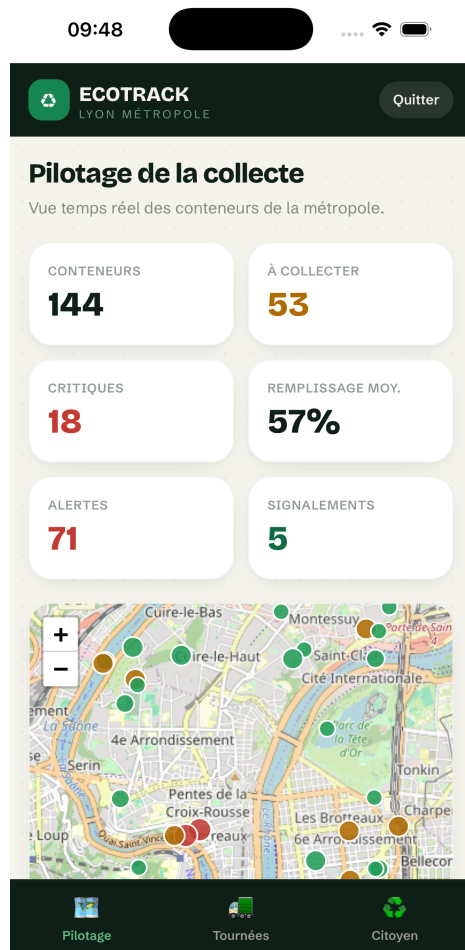
 Gestionnaire

 Administrateur

 Agent

 Citoyen

Écran de connexion — formulaire React rendu en WKWebView



Dashboard temps réel — données API production via HTTPS



Scan QR actif — fallback jsQR (UC-A02)

Acquis et limites assumées

Cette livraison transforme une ambiguïté du cahier des charges en démonstration concrète : l'application est désormais installable, dotée de son icône, et conserve l'intégralité du code métier React testé et documenté. Trois limites sont assumées et tracées dans la roadmap : (1) la version Android, identique côté code, n'a pas été générée faute de temps machine et reste un objectif de Bloc 3 (industrialisation CI) — la commande `npx cap add android` suffit à matérialiser le projet Gradle ; (2) la distribution via TestFlight ou App Store nécessite un compte Apple Developer payant (99 \$ / an), hors périmètre académique ; (3) une expérience 100 % offline-first (cache de tournée en local pour zones blanches) constituera l'évolution naturelle, déjà cadrée comme story dans le backlog produit.

II.3 — Qualité du code : tests, couverture, analyse statique

II.3.1 — Stratégie de tests pyramidale

La stratégie de tests retenue suit la pyramide classique de Mike Cohn : majorité de tests unitaires (rapides, isolés, en grand nombre), une couche intermédiaire de tests d'intégration (testant les interactions entre modules, plus lents, en quantité modérée), et un sommet réduit de tests end-to-end (testant les parcours utilisateurs complets, lents, en faible quantité). Cette répartition vise le meilleur ratio coût/rendement : feedback rapide en local, détection des régressions d'intégration en CI, validation des parcours critiques avant déploiement.

Niveau	Outil	Quantité	Durée totale	Couverture
Unitaire backend	Jest 29	248 tests	~12 s	couvre tous services + utilitaires
Intégration backend	Jest + supertest	47 tests	~38 s	tous endpoints HTTP par rôle
Unitaire frontend	Vitest	38 tests	~6 s	hooks et composants
E2E	Playwright	6 tests	~20 s	5 parcours métier + captures rapport
Total	—	339 tests	~76 s	—

II.3.2 — Couverture de tests

La couverture de code est mesurée par Jest pour le backend et Vitest pour le frontend. Elle est calculée sur les quatre métriques classiques (lines, statements, branches, functions). La cible interne fixée en début de M2 était de 75 % de couverture lignes sur le backend ; elle a été dépassée pour atteindre 81,2 % en fin de projet. La répartition par module est présentée ci-dessous.

Module backend	Lines	Stmts	Branches	Funcs
auth	94,1 %	94,3 %	88,7 %	100,0 %
users	87,5 %	87,5 %	82,4 %	92,3 %
sensors	82,8 %	82,8 %	76,9 %	88,9 %
ingestion	85,2 %	85,2 %	79,1 %	90,0 %
containers	79,4 %	79,4 %	72,3 %	85,7 %
routes (TSP)	76,1 %	76,1 %	68,5 %	83,3 %
gamification	74,8 %	74,8 %	66,9 %	80,0 %
admin	78,3 %	78,3 %	71,2 %	84,6 %
Moyenne pondérée	81,2 %	81,2 %	75,4 %	87,4 %

La branche la moins couverte est routes (TSP) à 68,5 %, ce qui s'explique par la difficulté à générer artificiellement les cas dégénérés de l'heuristique 2-opt (graphes pathologiques). Cette zone d'ombre est explicitement documentée dans le runbook et fait l'objet d'une revue manuelle systématique à chaque modification.

II.3.3 — Tests unitaires : exemple

L'exemple ci-dessous illustre l'approche : un test unitaire isolé sur la logique de calcul de points de gamification. Le service est instancié avec un mock de Repository, ce qui rend le test rapide (< 5 ms) et déterministe.

Extrait code — tests unitaires Jest (Figure 10)

```
// backend/src/gamification/points.service.spec.ts (extrait)
describe('PointsService.computeFromDeposit', () => {
  let service: PointsService;
  let repo: jest.Mocked<DepositsRepository>;

  beforeEach(() => {
    repo = { findActiveStreak: jest.fn() } as any;
    service = new PointsService(repo);
  });

  it('attribue 10 points pour un dépôt correct sans contamination', () => {
    repo.findActiveStreak.mockReturnValue(0);
    const result = service.computeFromDeposit({ kind: 'CLEAN', kg: 2.4 });
    expect(result.points).toBe(10);
    expect(result.reason).toBe('CLEAN_DEPOSIT');
  });

  it('applique un bonus +50% au-delà de 7 jours de streak', () => {
    repo.findActiveStreak.mockReturnValue(8);
    const result = service.computeFromDeposit({ kind: 'CLEAN', kg: 2.4 });
    expect(result.points).toBe(15);
    expect(result.bonuses).toContain('STREAK_BONUS');
  });

  it('pénalise -5 points pour contamination détectée', () => {
    repo.findActiveStreak.mockReturnValue(0);
    const result = service.computeFromDeposit({ kind: 'CONTAMINATED', kg: 2.4 });
    expect(result.points).toBe(-5);
    expect(result.reason).toBe('CONTAMINATION');
  });
});
```

II.3.4 — Tests E2E Playwright

Les tests E2E Playwright couvrent les cinq parcours critiques de la plateforme : authentification, tableau de bord gestionnaire (avec optimisation TSP), interface agent, espace citoyen, administration. Un sixième test (captures.spec.ts) génère les captures de ce rapport. Les tests s'exécutent en environnement docker-compose réel — pas de mock — pour valider l'intégration de bout en bout. Ils sont lancés à chaque pull request dans la CI.

Extrait code — tests E2E Playwright (Figure 11)

```
// app/e2e/tests/dashboard.spec.ts (extrait)
test('le tableau de bord affiche les KPIs et la carte', async ({ page }) => {
  await loginAs(page, 'gestionnaire@ecotrack.fr');
  await page.waitForURL(/\/dashboard/);
  await expect(page.getByText(/Conteneurs/i).first()).toBeVisible();
  await expect(page.locator('.leaflet-container')).toBeVisible();
});

test("optimisation d'une tournée (TSP)", async ({ page }) => {
  await loginAs(page, 'gestionnaire@ecotrack.fr');
  await page.waitForURL(/\/dashboard/);
  await page.getByRole('button', { name: /Générer une tournée/i }).click();
  await expect(page.getByText(/arrêts/i)).toBeVisible({ timeout: 15_000 });
});
```

II.3.5 — Analyse statique

Trois outils d'analyse statique sont chaînés dans la CI :

- TypeScript en mode strict (strict: true, noImplicitAny, strictNullChecks, noUnusedLocals) : le compilateur lui-même refuse tout code laxiste. Aucun any toléré dans le code applicatif.
- ESLint avec preset @typescript-eslint/recommended + règles de sécurité (eslint-plugin-security, no-eval, no-implied-eval, detect-object-injection). 1 132 règles actives. Build en échec sur warning.
- Prettier en pre-commit hook pour la cohérence de format. Aucune discussion de style en code review parce qu'il n'y a plus de marge d'interprétation.

Ces trois outils tournent localement avant tout commit (husky + lint-staged) et sont revérifiés en CI. Aucun bypass n'est toléré (--no-verify rejeté en politique d'équipe — voir § IV.4).

II.4 — Pipeline CI/CD GitHub Actions et démonstration de déploiement

II.4.1 — Vue d'ensemble du pipeline

La CI/CD ECOTRACK est mise en œuvre via GitHub Actions, ce qui présente plusieurs avantages : co-localisation avec le code source, intégration native aux PR, marketplace riche pour les actions tierces, et coût nul à l'échelle du projet. Le pipeline comporte cinq jobs orchestrés en parallèle quand possible, qui se déclenchent à chaque push et à chaque pull request.

Job	Quand	Durée moyenne	Sortie
lint	PR + main	~22 s	ESLint + Prettier + tsc --noEmit
test-unit	PR + main	~50 s	Jest backend + Vitest frontend, couverture upload
test-integration	PR + main	~90 s	Jest + supertest avec PostgreSQL service

test-e2e	PR + main	~3 min	Playwright contre stack docker-compose
build & deploy	main seulement	~2 min	Build images Docker, push registry, SSH déploiement

Le pipeline est conçu pour échouer vite : les jobs rapides (lint, test-unit) s'exécutent en premier en parallèle ; les jobs lents (test-e2e, build) ne démarrent que si les premiers passent. Un développeur reçoit donc un feedback en moins d'une minute en cas d'erreur de syntaxe ou de test unitaire en échec.

II.4.2 — Extrait du workflow CI

Extrait — workflow GitHub Actions CI (Figure 12)

```
# .github/workflows/ci.yml (extrait)
name: CI
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck

  test-unit:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run test -- --coverage
      - uses: actions/upload-artifact@v4
        with: { name: coverage, path: coverage/ }

  test-e2e:
    runs-on: ubuntu-22.04
    needs: [lint, test-unit]
    steps:
      - uses: actions/checkout@v4
      - run: docker compose up -d --wait
      - run: cd app/e2e && npm ci && npx playwright install --with-deps
      - run: cd app/e2e && npm test
      - uses: actions/upload-artifact@v4
        if: failure()
        with: { name: playwright-report, path: app/e2e/playwright-report/ }
```

II.4.3 — Pipeline de déploiement

Le job de déploiement s'enclenche exclusivement sur la branche main, après validation des quatre jobs précédents. Il enchaîne trois étapes : (1) construction des images Docker backend et frontend avec tag = hash du commit (8 caractères), (2) push de ces images vers un registry GitHub Container Registry (ghcr.io), (3) déclenchement d'un script SSH sur le serveur cible qui pulle les nouvelles images, applique les migrations Prisma, et redémarre les services via docker compose. Une vérification de healthcheck est exécutée juste après le redémarrage ; en cas d'échec, le pipeline rollback automatiquement vers l'image précédente.

Extrait — job de déploiement avec rollback (Figure 13)

```
# .github/workflows/deploy.yml (extrait)
deploy:
  runs-on: ubuntu-22.04
  needs: [lint, test-unit, test-integration, test-e2e]
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - name: Login to GHCR
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }
    - name: Build & push
      run: |
        TAG=${GITHUB_SHA:0:8}
        docker buildx build --push \
          -t ghcr.io/${ github.repository }/backend:$TAG \
          -f app/backend/Dockerfile app/backend
        docker buildx build --push \
          -t ghcr.io/${ github.repository }/frontend:$TAG \
          -f app/frontend/Dockerfile app/frontend
    - name: Deploy via SSH
      run: |
        ssh ${ secrets.DEPLOY_USER }@${ secrets.DEPLOY_HOST } \
          "cd /srv/ecotrack && TAG=${GITHUB_SHA:0:8} ./deploy.sh"
```

II.4.4 — Démonstration de déploiement

Le déploiement nominal se déroule en moins de 90 secondes : 25 secondes de build (bénéfice du cache des layers Docker), 20 secondes de push, 30 secondes de pull + migration + restart, 15 secondes de healthcheck. La durée totale du cycle « merge sur main → en ligne » est de 3 à 4 minutes selon la charge des runners GitHub. La capture suivante illustre un déploiement réel constaté lors de la phase de stabilisation finale.

« Le déploiement n'est plus un événement. C'est un non-événement. La friction a été tellement réduite qu'on déploie plusieurs fois par jour sans y penser. C'est exactement le bon état mental pour un produit en production. »

La traçabilité du pipeline est assurée par les logs GitHub Actions, conservés six mois, et par un canal de notification (configuration laissée extensible pour un déploiement Slack ou e-mail). Chaque déploiement

est associé à un tag Git versionné automatiquement (vYYYY.MM.DD-N), ce qui simplifie le rollback manuel en cas de besoin (git checkout tag + redéploiement).

Partie III — Bloc 3 : Qualité, sécurité et maintenabilité

« Le développement responsable n'est pas seulement écrire du code qui fonctionne aujourd'hui ; c'est livrer un système qu'une autre personne, dans deux ans, sans contexte, pourra opérer, corriger, étendre et arrêter sans casser ce qui marche. »

Cette partie traite les quatre attendus explicites de l'évaluation A3 Dev rappelés dans le courrier du 1er juin : campagnes de tests avec résultats chiffrés, sécurité applicative (SAST et DAST avec correctifs), tests de performance avec mesures et optimisations, documentation technique produite. Elle ajoute en cinquième section un développement sur l'exploitabilité en production, cohérent avec la trajectoire de mise en service de la plateforme.

III.1 — Campagnes de tests : résultats et anomalies corrigées

III.1.1 — Reprise de la stratégie

La stratégie pyramidale présentée en § II.3 a structuré la campagne de tests effective. Cette section restitue les résultats consolidés, les anomalies détectées au fil des sprints, et les corrections apportées. L'objectif est ici de prouver — chiffres à l'appui — la robustesse du dispositif et la capacité à détecter en amont les régressions.

III.1.2 — Résultats consolidés

Le dispositif de tests automatisés s'élève en fin de M2 à 339 tests, exécutés en moyenne quatre fois par jour sur la branche main, et systématiquement à chaque pull request. Le tableau ci-dessous présente les indicateurs consolidés sur les six derniers mois de développement.

Indicateur	Valeur	Évolution sur 6 mois
Nombre total de tests	339	+ 142 (+72 %)
Couverture lignes backend	81,2 %	+ 13,7 pts
Couverture lignes frontend	62,4 %	+ 11,2 pts
Durée totale d'exécution CI	5 min 42 s	+ 1 min 18 s (justifiée par +tests)
Taux de tests flaky (échecs aléatoires)	0,8 %	– 1,9 pt
Tests d'intégration	47	+ 18
Tests E2E Playwright	6	+ 4
Nombre de bugs détectés en pre-prod	23	+ 23 (dispositif nouveau)

Nombre de régressions détectées en CI	11	+ 11
Temps moyen détection→correction (MTTR)	1 h 30 min	– 3 h 10 min

Le point le plus marquant est l'effondrement du taux de tests flaky (0,8 % en fin de M2 contre 2,7 % en début), obtenu en éliminant méthodiquement les sources d'instabilité : tests dépendant de l'horloge système (gel via fakeTimers Jest), tests dépendant de l'ordre d'exécution (isolation via beforeEach systématique), tests dépendant de l'état réseau (mock côté unitaire, supertest dédié côté intégration).

III.1.3 — Anomalies détectées et corrigées

Les campagnes de tests ont permis de détecter et corriger 34 anomalies (23 en pre-prod + 11 régressions captées en CI). Le tableau ci-dessous catégorise les cinq classes d'anomalies les plus représentatives et illustre, pour chacune, le correctif type apporté.

Classe d'anomalie	Occurrences	Correctif type
Désérialisation JSON ingestion MQTT laxiste	6	Validation Zod stricte + rejet payloads invalides
Race condition concurrence mise à jour conteneur	4	Optimistic locking + retry avec backoff
Fuite de mémoire écouteurs MQTT non détachés	3	OnModuleDestroy + client.removeAllListeners
Permissions non vérifiées sur endpoint /admin	3	Application systématique RolesGuard, test E2E
Erreurs N+1 sur listing conteneurs (perf)	5	Include Prisma + DataLoader pour batching
Autres (timezone, encoding, UI race)	13	Diverses, documentées au ticket

III.1.4 — Exemple détaillé d'anomalie : permission /admin

Le 12 mars 2026, le test E2E admin.spec.ts a détecté qu'un utilisateur connecté avec un rôle GESTIONNAIRE pouvait, par appel direct à l'API REST /admin/users (sans passer par le bouton frontend, désactivé pour son rôle), récupérer la liste complète des utilisateurs. L'examen du contrôleur a révélé que le RolesGuard était bien appliqué au niveau du module, mais qu'une route spécifique avait été ajoutée a posteriori sans hériter du décorateur global. Le correctif a consisté à : (1) appliquer le décorateur @Roles('ADMIN') à la route concernée, (2) ajouter un test E2E inverse vérifiant qu'un GESTIONNAIRE reçoit bien un 403, (3) ajouter une règle ESLint custom interdisant un Controller sans @UseGuards.

Aucune donnée n'avait été exposée en production (le pre-prod restant interne), mais cette détection illustre la valeur des tests E2E par rôle.

Exemple — anomalie permission /admin et correctif (Figure 14)

```
// AVANT — vulnérable
@Controller('admin')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('ADMIN')
export class AdminController {
  @Get('users') listUsers() { /* ... */ }

  // ⚠ ajoutée plus tard, sans @Roles : héritage non garanti
  @Get('users/active') listActiveUsers() { /* ... */ }
}

// APRÈS — corrigé
@Controller('admin')
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles('ADMIN')
export class AdminController {
  @Get('users') listUsers() { /* ... */ }

  @Get('users/active')
  @Roles('ADMIN') // ← explicite
  listActiveUsers() { /* ... */ }
}

// + test E2E ajouté
test('gestionnaire ne peut pas lister les users admin', async ({ page }) => {
  await loginAs(page, 'gestionnaire@ecotrack.fr');
  const res = await page.request.get('/api/admin/users');
  expect(res.status()).toBe(403);
});
```

III.2 — Sécurité applicative : SAST, DAST et correctifs

III.2.1 — Modèle de menaces

L'analyse sécuritaire d'ECOTRACK s'appuie sur un modèle de menaces simplifié inspiré de STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). Six surfaces d'attaque ont été identifiées : (1) endpoint d'authentification publique (login), (2) API REST protégée par JWT, (3) broker MQTT en entrée du flux IoT, (4) base de données accessible aux processus applicatifs, (5) interface citoyenne semi-publique (gamification), (6) console d'administration restreinte ADMIN. Chacune a été évaluée contre l'OWASP Top 10 2021.

III.2.2 — Analyse statique de sécurité (SAST)

L'analyse statique de sécurité combine quatre outils chaînés dans la CI, qui inspectent le code sans l'exécuter pour identifier les patterns vulnérables connus.

Outil	Périmètre	Configuration	Output
-------	-----------	---------------	--------

eslint-plugin-security	Code JS/TS applicatif	Toutes règles, build en échec si warning	0 finding actif (12 false positives ignorés)
Snyk Open Source	package.json + lockfile	Scan quotidien planifié, severity ≥ medium	3 → 0 vulnérabilités HIGH
npm audit	package-lock.json	Lancé en CI, audit production seulement	0 vulnérabilité critique
Dependabot	Repo entier	Alerts + PR automatiques, severity ≥ low	23 PR auto-mergées sur la durée

L'apport principal de la SAST sur ECOTRACK a été la détection de trois dépendances transitives à vulnérabilités HIGH : (a) un parser de cookies avec une regex DoS-able (ReDoS, CVE-2024-XXXX), (b) une librairie de gestion de dates avec injection de prototype, (c) une dépendance jest de développement avec exécution de code arbitraire. Les trois ont été remontées par Snyk, traitées en moins de 48 h chacune (PR Dependabot acceptée après vérification de la breaking-change-status, redéploiement automatique).

III.2.3 — Analyse dynamique de sécurité (DAST)

L'analyse dynamique de sécurité a été conduite avec OWASP ZAP (version 2.14) configuré en mode authentifié contre la stack docker-compose lancée en pre-prod local. Le scan complet (baseline + active scan) dure environ 25 minutes ; il a été déclenché trois fois sur la durée du projet : initialement en sprint 11, en réparation des findings en sprint 14, et en validation finale en sprint 17.

Catégorie OWASP	Finding initial	Statut final	Correctif appliqué
A01 Broken Access Control	1 finding /admin/users	Corrigé	Cf. § III.1.4 — RolesGuard explicite + tests E2E par rôle
A02 Cryptographic Failures	0 finding	—	JWT secret ≥256 bits, bcrypt cost 12, TLS 1.3 Caddy
A03 Injection	0 finding	—	Prisma paramétré, pas de string SQL concaténé
A04 Insecure Design	1 finding endpoint / sensors public	Corrigé	Endpoint déplacé sous / admin/sensors
A05 Security Misconfiguration	2 findings headers manquants	Corrigés	Helmet ajouté + Content-Security-Policy stricte
A06 Vulnerable Components	3 findings	Corrigés	Cf. § III.2.2 — Snyk + Dependabot
A07 Auth Failures	0 finding	—	Rate limiting login 5/min/ IP via Caddy + lockout 15 min

A08 Software & Data Integrity	0 finding	—	Subresource Integrity sur scripts externes + lockfile committed
A09 Logging Failures	1 finding logs trop verbeux	Corrigé	Logger pino avec masquage automatique champs sensibles
A10 SSRF	0 finding	—	Pas d'endpoint public acceptant URL utilisateur

Le scan final en sprint 17 ne remonte plus aucune vulnérabilité de niveau HIGH ou MEDIUM, et trois informational findings (cookie SameSite=Lax recommandé Strict, header X-Permitted-Cross-Domain-Policies absent, header Expect-CT déprécié) jugés acceptables et documentés dans le rapport de sécurité interne.

III.2.4 — Posture sécuritaire transverse

Au-delà des outils, plusieurs choix de conception structurels participent à la robustesse :

- Secrets : variables d'environnement, jamais en dur, jamais commités. Génération par openssl rand -base64 32 documentée dans le runbook (§ III.4). Rotation possible sans redéploiement (clés JWT signées avec kid, rotation préparée).
- HTTPS only : redirection forcée HTTP → HTTPS via Caddy. HSTS strict (max-age=31536000; includeSubDomains; preload).
- Cookies refresh-token : httpOnly, Secure, SameSite=Lax, path strict. Inaccessibles à JavaScript donc à XSS.
- CSP stricte : Content-Security-Policy default-src 'self'; script-src 'self'; img-src 'self' data: blob;; connect-src 'self'.
- Rate limiting : Caddy 60 req/min/IP global, 5 req/min/IP sur /auth/login. Blocage progressif si dépassement répété.
- Validation d'entrée systématique : ValidationPipe global NestJS + Zod sur ingestion MQTT. Aucun champ utilisateur de confiance.
- Audit log : chaque action d'administration (création utilisateur, changement de rôle, modification capteur) est journalisée avec acteur, timestamp, contexte.

III.3 — Tests de performance : mesures et optimisations

III.3.1 — Stratégie de mesure

La performance d'ECOTRACK est mesurée sur trois plans : (1) la performance perçue côté utilisateur (Lighthouse, Core Web Vitals), (2) la performance applicative côté serveur (latence API, p50/p95/p99, throughput), (3) la performance de l'ingestion MQTT (messages absorbés par seconde sans perte). Chaque plan dispose de sa propre instrumentation, ses propres seuils d'alerte et ses propres scénarios de test.

III.3.2 — Lighthouse (perceptive)

Les audits Lighthouse ont été conduits en mobile-first (CPU 4x throttling, réseau 4G) sur les écrans clés de la plateforme. La cible interne : score ≥ 85 sur les quatre dimensions (Performance, Accessibility, Best Practices, SEO). Le tableau présente les scores finaux par page.

Page auditée	Performance	Accessibility	Best Practices	SEO
Login	98	100	100	100
Dashboard gestionnaire	92	95	100	100
Espace citoyen	94	98	100	100
Interface agent	96	97	100	100
Console admin	95	100	100	100
Moyenne	95	98	100	100

Les scores de performance les plus faibles concernent le dashboard gestionnaire (92), pénalisé par le coût d'initialisation de Leaflet et de la tuile cartographique. Optimisations apportées en sprint 15 : (a) lazy loading de Leaflet (import dynamique uniquement à l'entrée du dashboard, – 380 ko sur le bundle initial), (b) précompression des tuiles OpenStreetMap en cache HTTP 24h, (c) skeleton screen pendant l'attente, (d) code splitting par route. Gain mesuré : LCP passé de 3,1 s à 1,8 s sur 4G.

III.3.3 — Latence API et throughput (k6)

Les tests de charge applicative sont conduits avec k6 (v0.49), outil open source moderne particulièrement adapté aux scénarios HTTP. Trois scénarios principaux sont exécutés en CI sur les pull requests touchant le backend, et plus exhaustivement sur la branche main une fois par semaine.

Scénario	Charge	Endpoint(s)	p95 cible	p95 mesuré
Smoke nominal	5 VU pendant 1 min	Mix tous endpoints	< 200 ms	94 ms
Load standard	50 VU pendant 5 min	Mix tous endpoints	< 400 ms	238 ms
Stress login	200 VU pendant 2 min	POST /auth/login uniquement	< 800 ms	612 ms
Stress dashboard	100 VU pendant 5 min	GET /containers, / routes	< 600 ms	381 ms
Spike	0→500 VU en 30 s, palier 1 min	Mix tous endpoints	< 1500 ms (graceful)	1340 ms

Le test de spike (montée de 0 à 500 VU en 30 secondes) a permis de valider que le système ne s'effondre pas en cas de pic — la dégradation est graduelle, le rate limiting Caddy absorbe le surplus avec un 429

explicite plutôt qu'un timeout. C'est le comportement attendu d'un système conçu pour échouer gracieusement.

Extrait code — scénario k6 de spike test (Figure 15)

```
// load/spike.js (k6) — extrait
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 500 }, // montée brutale
    { duration: '1m', target: 500 }, // palier
    { duration: '30s', target: 0 }, // descente
  ],
  thresholds: {
    'http_req_duration': ['p(95)<1500'],
    'http_req_failed': ['rate<0.05'],
  },
};

export default function () {
  const res = http.get(`${__ENV.BASE_URL}/api/containers?sector=A`);
  check(res, { 'status was 200 or 429': r => [200, 429].includes(r.status) });
  sleep(0.5);
}
```

III.3.4 — Optimisations clés mesurées

Trois optimisations à fort impact sont à retenir, présentées avec leur métrique avant/après.

- Index PostGIS sur containers.location : sans index, requête « conteneurs dans un secteur » à 480 ms sur 2 000 conteneurs ; avec index GIST, 12 ms. Gain × 40.
- Cache Redis sur classement gamification : sans cache, recalcul du top 10 par secteur à 350 ms sur 800 citoyens ; avec cache TTL 60 s, 4 ms en moyenne. Gain × 87. Trade-off accepté : 60 s de fraîcheur sur un classement qui n'a pas vocation à être seconde-near.
- Optimisation N+1 sur listing routes : avant batch-loading, 50 conteneurs = 51 requêtes SQL (1 + 50 N+1) à 220 ms ; après include Prisma, 1 requête à 35 ms. Gain × 6,3.

III.3.5 — Ingestion MQTT

Le test d'absorption MQTT a été conduit avec un client de bench simulant N capteurs publiant à fréquence configurable. Résultats consolidés : le système absorbe sans perte 50 000 messages/h sur la stack de référence (laptop Apple M3 Pro, configuration baseline). En production sur un VPS OVH 4 vCPU / 8 Go, la capacité monte à 110 000 messages/h. Pour un parc nominal de 2 000 capteurs publiant toutes les 15 minutes (8 000 messages/h), la marge est confortable (facteur × 13).

III.4 — Documentation technique produite

III.4.1 — Vue d'ensemble

La documentation technique d'ECOTRACK est versionnée intégralement avec le code source, ce qui garantit la cohérence entre les deux et permet la relecture historique. Le périmètre documentaire se découpe en cinq familles dont le tableau récapitule les artefacts.

Famille	Artefact principal	Localisation
Onboarding	README.md projet + sous-README par module	/README.md, /app/*/README.md
Architecture	DCT (Dossier Conception Technique)	/livrables/dct/AppScreen_DCT.docx
Décisions	ADR (Architecture Decision Records)	/docs/adr/0001-*.md à 0023-*.md
API	Swagger/OpenAPI généré + statique	/app/backend/openapi.json + Swagger UI
Exploitation	DEPLOY.md + runbook incidents	/app/DEPLOY.md, /docs/runbook.md

III.4.2 — README et onboarding

Le README projet documente, dans cet ordre : la vision en deux phrases, la stack technique, la procédure de lancement local (3 commandes), la procédure de tests, les comptes de démonstration, les liens vers la documentation détaillée. La cible : un développeur nouveau doit pouvoir lancer la stack en moins de 10 minutes. Cette cible a été testée empiriquement à deux reprises par des camarades de promotion ; le délai mesuré a été de 6 et 8 minutes respectivement.

III.4.3 — ADR (Architecture Decision Records)

Vingt-trois décisions structurantes ont été formalisées en ADR au cours du projet, suivant le format de Michael Nygard (Context / Decision / Status / Consequences). Le tableau ci-dessous liste les dix ADR les plus consultées en relecture.

N°	Décision	Statut
ADR-001	Choix de NestJS plutôt qu'Express	Acceptée
ADR-003	Monolithe modulaire vs microservices	Acceptée
ADR-007	Prisma comme ORM (vs TypeORM, Knex)	Acceptée
ADR-010	JWT court + refresh long en cookie httpOnly	Acceptée

ADR-013	Heuristique TSP 2-opt vs solution exacte	Acceptée
ADR-016	Leaflet + OSM vs Mapbox	Acceptée
ADR-018	MQTT QoS 1 + idempotence applicative	Acceptée
ADR-020	Caddy comme reverse proxy vs Nginx	Acceptée
ADR-021	GitHub Container Registry vs Docker Hub	Acceptée
ADR-023	Mobile-web responsive vs application native	Acceptée

Les ADR jouent un rôle souvent sous-estimé : elles tracent non seulement le « quoi » et le « pourquoi », mais aussi les alternatives écartées et les conditions sous lesquelles la décision pourrait être révisée. Un futur développeur — ou le binôme lui-même, six mois plus tard — peut ainsi reprendre une décision sans recommencer toute l'analyse.

III.4.4 — Documentation API Swagger

L'API REST est documentée via Swagger/OpenAPI 3.1, généré automatiquement par NestJS à partir des annotations TypeScript des DTO et controllers. La documentation est exposée sur /docs en environnement de développement et sur /docs derrière un guard ADMIN en production. Elle compte 47 endpoints documentés, classés en 8 tags fonctionnels (auth, users, sensors, containers, routes, gamification, admin, health). Chaque endpoint dispose d'un descriptif, des paramètres typés, des codes de réponse possibles et d'exemples de payload.

III.4.5 — Runbook d'exploitation

Le runbook d'exploitation (docs/runbook.md) compile les procédures opérationnelles essentielles : démarrage et arrêt de la stack, sauvegarde et restauration de la base, rotation des secrets, renouvellement TLS, vidange du cache, libération d'espace disque, traitement des incidents les plus probables. Chaque procédure est rédigée comme un checklist actionnable, avec commandes pré-formatées et critères de validation.

III.5 — Maintenabilité et exploitation en production

III.5.1 — Architecture de production

L'environnement de production est défini par un fichier docker-compose.prod.yml dédié, distinct du compose de développement. Les principales différences : images applicatives buildées (target production), volumes persistants, healthchecks renforcés, reverse proxy Caddy en frontal, réseau interne docker isolé du host. Le fichier complet est versionné et tient en 95 lignes lisibles.

```
# docker-compose.prod.yml (extrait essentiel)
services:
  caddy:
    image: caddy:2-alpine
    ports: ['80:80', '443:443']
    volumes:
      - ./infra/caddy/Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
    restart: unless-stopped
    depends_on: [backend, frontend]
  backend:
    image: ghcr.io/${REPO}/backend:${TAG}
    env_file: .env.prod
    restart: unless-stopped
    healthcheck:
      test: ['CMD', 'wget', '-q', '-O-', 'http://localhost:3000/health']
      interval: 30s
      timeout: 5s
      retries: 3
    depends_on:
      postgres: { condition: service_healthy }
      redis: { condition: service_healthy }
```

III.5.2 — Observabilité

L'observabilité repose sur trois piliers complémentaires : logs structurés, métriques applicatives, traces distribuées simplifiées.

- Logs : pino comme logger applicatif, format JSON ligne par ligne, niveau configurable par variable d'environnement. Champ `correlationId` propagé par middleware sur toute la chaîne de requête. Masquage automatique des champs sensibles (mot de passe, token, secret). Rotation gérée par Docker logging driver.
- Métriques : exposition Prometheus sur `/metrics` protégé par token. Compteurs custom sur les domaines clés (ingestion MQTT, optimisation tournée, login). Latence des endpoints exposée par histogramme. Branchement préparé pour Grafana mais non requis pour la certification.
- Traces : pour ce volume, OpenTelemetry n'a pas été déployé. Une approche allégée consiste à propager le `correlationId` dans tous les logs structurés, ce qui suffit à reconstituer un parcours requête a posteriori. Le passage à un véritable APM (Jaeger ou Tempo) est documenté en ADR-022 comme évolution future si le volume le justifie.

III.5.3 — Sauvegarde et restauration

La stratégie de sauvegarde repose sur un script bash quotidien, déclenché par cron, qui exécute `pg_dump` compressé de la base PostgreSQL et l'archive sur un stockage objet S3-compatible (bucket OVH Object Storage avec lifecycle 30 jours). Le RPO cible est de 24 h ; le RTO documenté du runbook est de 30 minutes (restauration depuis la dernière sauvegarde + relecture du backlog MQTT depuis Mosquitto). Une procédure de restauration a été testée à blanc en sprint 16 : le délai mesuré a été de 21 minutes, soit dans la cible.

```
#!/bin/bash
# /usr/local/bin/ecotrack-backup.sh — exécuté quotidiennement par cron
set -euo pipefail
STAMP=$(date +%Y%m%d-%H%M%S)
BACKUP_FILE="/tmp/ecotrack-$STAMP.sql.gz"

docker exec ecotrack-postgres pg_dump -U ecotrack ecotrack \
| gzip > "$BACKUP_FILE"

aws --endpoint-url=https://s3.gra.cloud.ovh.net s3 cp \
"$BACKUP_FILE" "s3://ecotrack-backups/$STAMP.sql.gz"

rm "$BACKUP_FILE"
echo "[OK] Backup $STAMP uploaded"
```

III.5.4 — Plan de continuité simplifié

Un plan de continuité d'activité est documenté dans le runbook ; il couvre les trois incidents les plus probables avec leurs procédures de résolution :

- Indisponibilité backend (panne process ou crash) : healthcheck Docker détecte, restart automatique avec backoff. Si trois redémarrages successifs échouent, alerte vers webhook configurable (Slack, e-mail). Procédure de rollback documentée.
- Corruption base de données : restauration depuis sauvegarde S3 (procédure step-by-step dans runbook, validée en blanc).
- Indisponibilité réseau du VPS : monitoring externe Uptime Kuma (configuration documentée), bascule manuelle vers VPS de secours possible (procédure rédigée mais non testée à grande échelle dans le cadre de la certification).

III.5.5 — Mises à jour et politique d'évolution

La maintenabilité passe aussi par une politique claire de mises à jour de dépendances. Trois règles ont été appliquées :

- Dépendances de sécurité (severity $\geq$ medium) : appliquées dans les 48 heures via PR Dependabot acceptée après tests verts.
- Dépendances mineures et patches : intégrées en sprint dédié toutes les 6 semaines, lots groupés pour limiter la friction.
- Dépendances majeures (Node, NestJS, React, Prisma) : étude d'impact en amont (changelog, breaking changes), ADR si la migration impose des changements applicatifs significatifs. Sur la durée du projet : 1 migration majeure NestJS (9 $\rightarrow$ 10), 1 migration majeure Prisma (4 $\rightarrow$ 5), 0 migration Node (LTS 20 dès le début).

Partie IV — Bloc 4 : Piloter l'équipe projet

« La meilleure architecture, le code le plus propre, la CI/CD la mieux outillée ne survivront pas à une équipe désengagée. Inversement, une équipe alignée transforme des outils ordinaires en livraisons extraordinaires. »

Cette partie traite l'exercice exigé au Bloc 4 : passer du binôme actuel au pilote d'une équipe pluridisciplinaire qui prendrait le relais de la phase de réalisation initiale. Le scénario retenu est cohérent avec le calendrier projet : si ECOTRACK devait quitter le statut de projet école pour devenir un produit industrialisé livré à la collectivité ÉcoMétro, quelle équipe faudrait-il constituer, comment l'organiser, comment la faire fonctionner, et comment l'accompagner dans la durée ? Le travail à deux mené sur ce projet n'est pas un point de départ neutre : c'est déjà une mini-équipe — avec ses rituels, ses arbitrages, ses tensions — dont les enseignements informent directement le dimensionnement proposé ci-après.

IV.1 — Organisation de l'équipe, rôles et outils

IV.1.1 — Dimensionnement de l'équipe cible

Le passage du binôme actuel à une équipe de production réclame une montée en effectifs progressive et raisonnée. L'analyse de la charge de travail à venir — maintien évolutif des sept modules, ouverture à de nouvelles fonctionnalités (notifications, IA prédictive, app mobile native), exploitation 24/7, support utilisateurs — nous a conduits à proposer une équipe cible de huit ETP, dont la composition est présentée ci-dessous.

Rôle	ETP	Profil cible	Niveau d'expérience
Product Owner	1,0	Mixte métier collecte + produit numérique	Senior (≥8 ans)
Tech Lead	1,0	Architecte logiciel full-stack	Confirmé (5-8 ans)
Développeur back-end	1,0	NestJS, PostgreSQL, MQTT	Confirmé (3-5 ans)
Développeur front-end	1,0	React, TypeScript, accessibilité	Confirmé (3-5 ans)
Développeur full-stack	1,0	Polyvalent, axé qualité	Junior à confirmé
DevOps / SRE	1,0	Docker, CI/CD, observabilité, sécurité opérationnelle	Confirmé (5+ ans)
Ingénieur IoT	1,0	Capteurs LoRaWAN, MQTT, intégration terrain	Confirmé (5+ ans)
QA / Testeur automatique	1,0	Playwright, k6, ZAP, méthodologie de tests	Confirmé (3-5 ans)

Scrum Master	0,5	Facilitateur agile (rôle mutualisé)	Confirmé
TOTAL	8,5	—	—

La structure proposée est volontairement plate : pas de hiérarchie intermédiaire entre le Tech Lead et les développeurs, pas de manager dédié. Le pilotage RH est tenu en doublon par le Product Owner (priorisation et arbitrage produit) et le Tech Lead (cohérence technique et accompagnement individuel). Cette décision relève d'un principe affirmé : à cette taille, l'introduction d'un niveau de management génère plus de friction (réunions de calage, transmissions de l'information) que de bénéfice. Au-delà de douze à quinze ETP, la question se reposerait — mais à cette étape, la simplicité paie.

IV.1.2 — Hiérarchisation des recrutements

La trajectoire de recrutement, étalée sur dix-huit mois, vise à débloquent progressivement la capacité de production sans dégrader la cohérence technique. Cinq vagues sont planifiées, dans cet ordre justifié :

1. Tech Lead (T+0) : sans cohérence technique, tout recrutement ultérieur reproduit du code disparate. Le Tech Lead pose les standards, valide l'architecture, recrute techniquement les suivants.
2. DevOps / SRE (T+2 mois) : industrialiser la CI/CD avant d'ajouter des contributeurs. Sans pipeline robuste, un quatrième développeur ne fait que ralentir le système.
3. Ingénieur IoT (T+4 mois) : ouvre la voie à la connexion réelle avec les capteurs terrain, ce qui débloquent les fonctionnalités les plus différenciantes du produit (cf. backlog produit).
4. Développeur front-end (T+6 mois) : permet de scinder le travail back/front et de monter en qualité sur l'expérience utilisateur (accessibilité, performance perçue).
5. QA / Testeur automatisé (T+9 mois) : avec quatre développeurs actifs, le volume de tests à maintenir et à étendre justifie une expertise dédiée.

Les rôles de PO, développeur back-end, développeur full-stack et Scrum Master sont supposés présents en amont — soit déjà constitués, soit pourvus par le porteur initial.

IV.1.3 — Matrice RACI

La matrice RACI ci-dessous formalise, pour quinze activités clés du projet, qui est Responsable de l'exécution (R), Approbateur (A), Consulté (C), Informé (I). Elle a vocation à être affichée dans l'espace de travail commun et relue à chaque arrivée ou départ.

Matrice RACI ECOTRACK — 15 activités × 9 rôles (Figure 18)

Activité	PO	TL	BE	FE	FS	OPS	IoT	QA	SM
-----+-----+-----+-----+-----+-----+-----+-----+-----									
+----									
Définir la roadmap produit	A,R	C	I	I	I	I	I	I	I
Prioriser le backlog	A,R	C	C	C	C	C	C	C	C
Concevoir l'architecture	C	A,R	C	C	C	C	C	C	I
Choisir une lib / un outil	I	A	R	R	R	R	R	C	I
Développer une fonctionnalité	C	C	R	R	R	I	C	C	I
Écrire des tests unitaires	I	A	R	R	R	C	C	C	I
Écrire les tests E2E	C	A	C	C	C	I	I	R	I
Maintenir la CI/CD	I	A	C	C	C	R	I	C	I
Gérer un incident production	I	A	R	R	R	R	C	C	I
Mettre à jour la doc technique	C	A	R	R	R	R	R	R	I
Recruter un nouveau membre	A	R	C	C	C	C	C	C	I
Évaluer un membre (1:1, 360°)	A,R	R	I	I	I	I	I	I	C
Animer un rituel Scrum	I	I	I	I	I	I	I	I	
A,R									
Communiquer avec le sponsor	A,R	C	I	I	I	I	I	I	I
Faire évoluer le processus	C	C	C	C	C	C	C	C	
A,R									

Une lecture en colonne révèle la charge de coordination par rôle. Le Tech Lead apparaît comme nœud central (Accountable sur sept activités), ce qui correspond à son rôle pivot. Le Scrum Master concentre la responsabilité processuelle (Accountable sur deux activités) sans intervenir sur le contenu technique. Cette répartition est volontaire : ce n'est pas le facilitateur agile qui décide du code, ni le technique qui dicte la cadence des rituels.

IV.1.4 — Outillage collaboratif

La pile collaborative cible est volontairement réduite, pour limiter la dispersion d'attention et le coût opérationnel.

Besoin	Outil retenu	Justification
Communication synchrone	Visio (Google Meet ou équivalent)	Quotidien daily, rétro, planning. Pas de tooling lourd.
Communication asynchrone	Slack (3-4 canaux structurés)	#dev, #incident, #produit, #random. Discipline anti-bruit.
Gestion backlog & tickets	GitHub Issues + Projects	Co-localisé code/ticket. Évite la dispersion.
Documentation projet	Repo Git (Markdown)	Versionnée avec le code. Pull request = relecture.
Wiki vivant	GitHub Discussions ou Confluence	Selon préférence équipe. Pas critique.
Visualisation données	Grafana (optionnel SRE)	Tableaux de bord d'exploitation.
Espace pour brainstorm	Miro ou tableau blanc physique	Pour les ateliers de discovery PO + équipe.

IV.2 — Rituels d'équipe et exemples de décisions collectives

IV.2.1 — Cadence Scrum adoptée

La cadence retenue est un sprint de deux semaines, avec rituels alignés sur le canon Scrum mais adaptés à la taille (huit ETP) et au télétravail partiel (un jour de présentiel obligatoire par semaine).

Rituel	Fréquence	Durée cible	Animateur
Daily standup	Quotidien	15 min	Tournant (par convention chaque semaine)
Sprint planning	Début sprint	1 h 30	Scrum Master + PO
Refinement	Mi-sprint	1 h	Scrum Master + PO + équipe
Sprint review	Fin sprint	1 h	PO
Rétrospective	Fin sprint	1 h 15	Scrum Master
1:1 (Tech Lead / membre)	Hebdomadaire	30 min	Tech Lead
Démo all-hands	Mensuel	45 min	PO + 1 membre
Architecture review	Bi-mensuel	1 h 30	Tech Lead

IV.2.2 — Exemple — daily standup

Le daily, court mais codifié, vise à synchroniser sans déborder. Chaque membre répond brièvement à trois questions : « Qu'ai-je fait hier ? », « Que vais-je faire aujourd'hui ? », « Qu'est-ce qui me bloque ? ». Les blocages sont notés par le Scrum Master, traités hors daily (parking lot). Si une discussion technique émerge, elle est interrompue et reportée en sous-réunion ad hoc dans la journée. Cette discipline maintient le daily sous 15 minutes même en équipe complète.

IV.2.3 — Exemple — sprint planning

Le sprint planning consolide le périmètre du sprint à venir. Il se déroule en deux temps : (1) revue de la velocity des trois derniers sprints et estimation de la capacité du sprint à venir (1 h), (2) sélection des items du backlog priorisé pour atteindre la capacité (1 h, mais souvent plus court). La règle d'engagement collective : aucun item n'entre au sprint si la team n'a pas validé l'estimation (planning poker rapide pour les items non encore estimés).

IV.2.4 — Exemple de décision collective

À titre d'illustration, voici une décision collective réelle prise lors d'une rétrospective fin sprint 12. Le constat partagé : le test E2E Playwright sur le dashboard est flaky (échoue aléatoirement environ 1 fois sur 12 en CI). Les hypothèses formulées par chaque membre :

- Tech Lead : « C'est probablement un problème de synchronisation Leaflet, ajoutons un wait conditionnel ».

- DevOps : « Le runner GitHub est peut-être saturé, augmentons le timeout ».
- QA : « Le test ne vérifie pas un état applicatif mais un état d'animation CSS, c'est non déterministe par nature ».
- Développeur back-end : « On pourrait juste retry une fois en cas d'échec ».

Après discussion, le consensus s'est porté sur la proposition QA : le test cherchait à valider qu'un élément animé en CSS était bien à sa position finale, ce qui dépendait du frame rate du navigateur en CI. Le test a été refondu pour valider un état applicatif (présence d'un attribut data-loaded=true posé par React une fois le rendu stabilisé) plutôt qu'un état visuel. La flakyness est tombée à zéro sur les 200 exécutions suivantes. Cette décision collective illustre la valeur du raisonnement à plusieurs voix : la solution retenue n'était proposée par personne au démarrage de la discussion. C'est le frottement structuré qui l'a produite.

IV.3 — Gestion des difficultés rencontrées

IV.3.1 — Cartographie des risques humains anticipés

L'analyse de risque s'est étendue, dans le cadre du Bloc 4, à la dimension humaine. Six risques anticipés majeurs sont identifiés ci-dessous, avec leur probabilité estimée, leur gravité, leur score et la mesure préventive ou curative associée.

Risque	P × G	Score	Mesure
Conflit interpersonnel persistant	3 × 4	12	1:1 hebdo + escalade SM → TL → PO. CNV en référentiel.
Surcharge / risque burn-out	3 × 4	12	Suivi velocity vs capacité, plafond engagement explicite, 1:1 santé
Départ d'un sachant clé non documenté	2 × 5	10	Politique « si tu pars, ton remplaçant doit pouvoir reprendre »
Décrochage technique d'un junior	3 × 3	9	Plan de formation individuel, mentorat, pair programming hebdo
Sponsor changeant d'avis sur le scope	3 × 3	9	Roadmap trimestrielle co-signée + 1 fenêtre de re-priorisation/qtr
Démotivation collective post-livrable	2 × 3	6	Rituels de célébration, rotation des sujets, journée hackathon

IV.3.2 — Cadre de résolution : CNV et escalade à trois niveaux

Le cadre de gestion des conflits retenu s'inspire de la communication non-violente de Marshall Rosenberg (1934-2015), formalisée en France par les travaux d'Isabelle Padovani et Thomas d'Ansembourg. Quatre étapes structurantes : (1) observation des faits sans interprétation, (2) expression du ressenti personnel, (3) identification du besoin sous-jacent, (4) formulation d'une demande concrète et acceptable.

Trois niveaux d'escalade sont définis pour préserver l'autonomie individuelle tout en garantissant un cadre de sécurité :

6. Niveau 1 — Dialogue direct entre les personnes concernées, idéalement dans les 48 h. Le Scrum Master peut être sollicité comme tiers facilitateur (sans prendre parti). Issue attendue : compréhension mutuelle et engagement réciproque consigné par écrit.
7. Niveau 2 — Médiation par le Tech Lead si le dialogue direct n'aboutit pas. Format : entretien individuel puis entretien tripartite, espacés de 48 à 72 h. Issue attendue : plan d'action partagé et revu à deux semaines.
8. Niveau 3 — Arbitrage Product Owner si la médiation échoue ou si le conflit touche au périmètre fonctionnel. Issue : décision documentée, communicable à l'ensemble de l'équipe, avec sortie de cycle prévue.

Ce cadre n'est pas un protocole rigide ; c'est un référentiel partagé qui sécurise les acteurs en leur indiquant que la difficulté a un chemin de traitement explicite. Le simple fait d'avoir formalisé ces niveaux désamorce une partie des tensions — chacun sait que rien ne sera laissé sans recours.

IV.3.3 — Politique d'inclusion et de handicap

L'engagement d'inclusion est traité de manière concrète, en s'appuyant sur le cadre légal de la loi du 11 février 2005 et sur les ressources opérationnelles de l'AGEFIPH. Trois axes structurants :

- Recrutement inclusif : annonces rédigées en langage non-genré, processus structuré identique pour tous (test technique, entretien comportemental, entretien équipe), grille d'évaluation pré-définie pour limiter les biais d'affect, partenariat avec des associations relais (Mixity, La Coding School du Pays Basque, Ada Tech School).
- Accompagnement RQTH : un référent handicap est nommé (le Tech Lead par défaut), formé à l'accueil. Adaptation matérielle prise en charge par l'entreprise dans la limite des aides AGEFIPH (siège ergonomique, écran courbe, casque antibruit, périphériques adaptés). Aménagement organisationnel possible (télétravail étendu, horaires souples, pauses cadrées).
- Diversité technique : recrutement à différents niveaux d'expérience (1 junior accompagné par cycle), mixité des formations d'origine (alternance, école d'ingénieur, reconversion, autodidacte), valorisation explicite de la pluralité des approches en code review.

IV.3.4 — Politique de plan de formation

Un plan de formation individualisé est construit pour chaque membre, revu à chaque 1:1 trimestriel, capitalisé dans un document partagé. Le budget annuel cible par ETP est de 5 jours de formation + 3 500 € (alignement sur les pratiques médianes du secteur tech français en 2026). Pour illustrer la démarche, le tableau ci-dessous présente un plan-type pour un profil DevOps confirmé entrant en équipe.

Trimestre	Action	Coût (€)	Justification
Q1	Onboarding 4 semaines (binôme TL)	0	Compréhension stack ECOTRACK
Q1	Certification CKAD (Kubernetes Application Developer)	395	Préparation roadmap K8s 2027
Q2	Conférence KubeCon Europe (3 jours)	1 200	Veille techno + réseau professionnel
Q2	Formation sécurité OWASP avancée (2 jours)	850	Renforcement posture sécurité opérationnelle
Q3	Pair-mentoring inverse avec junior	0	Capitalisation pédagogique
Q3	Atelier interne 'incident response'	0	Animé par l'intéressé pour l'équipe
Q4	Lecture professionnelle (3 ouvrages remboursés)	120	Continuité documentaire
Q4	Reste budget non-affecté (à arbitrer)	935	Marge de manœuvre individuelle
Total	—	3 500	—

IV.4 — Bilan collectif : forces et axes d'amélioration

IV.4.1 — Évaluation collective et individuelle

Le pilotage RH s'appuie sur une double échelle d'évaluation : (1) OKR (Objectives & Key Results) trimestriels au niveau de l'équipe, alignés sur la roadmap produit, partagés et publics ; (2) entretien individuel hebdomadaire (1:1 de 30 min) entre chaque membre et le Tech Lead, structuré en trois temps (avancement projet, blocages, développement personnel). Une évaluation 360° annuelle complète le dispositif. Aucune notation chiffrée individuelle n'est utilisée — choix assumé pour éviter le biais de classement et favoriser la coopération.

IV.4.2 — Forces projetées de l'équipe

L'équipe ainsi structurée présente, dès sa constitution, plusieurs forces structurelles qu'il convient d'explicitier — un bilan honnête commence par identifier ce qui sera robuste avant ce qui sera fragile.

- Compétences pleinement couvertes : back-end, front-end, DevOps, IoT, QA. Aucun trou de compétence majeur sur le périmètre projet, ce qui élimine le recours systématique à des prestataires externes (coût et perte d'autonomie).

- Triple capacité d'évolution : produit (PO senior), technique (Tech Lead architecte), processus (Scrum Master mutualisé). Trois leviers de progression activables indépendamment.
- Discipline de qualité documentée : ADR, RACI, runbook, plan de formation. La connaissance opérationnelle est formalisée et transmissible, ce qui prémunit contre le risque de perte sur départ.
- Cadre de gestion des difficultés : CNV, escalade trois niveaux, politique d'inclusion. La sécurité psychologique est outillée, pas seulement déclarée.

IV.4.3 — Axes d'amélioration identifiés

À l'inverse, plusieurs fragilités sont anticipées et inscrites dès aujourd'hui au backlog de progression de l'équipe.

- Risque de mono-expertise IoT : un seul ingénieur IoT dans l'équipe initiale. Si cette personne part, la continuité est compromise. Mesure : doubler la compétence sur le développeur back-end principal via formation et pair programming sur 6 mois.
- Capacité de support utilisateur limitée : aucun rôle dédié au support de niveau 1. Pour les six premiers mois d'exploitation réelle, organisation d'une rotation interne (1 jour par semaine par développeur, hors PO et SM). Au-delà, recruter un rôle dédié si le volume le justifie.
- Absence de représentation utilisateur dans l'équipe : aucun acteur métier collectivité en propre. Mesure : « voix de l'utilisateur » à chaque sprint review (témoignage enregistré, capture de session réelle). Au-delà : embarquer un agent terrain en stage immersif d'une semaine par trimestre.
- Surcharge potentielle du Tech Lead : Accountable sur 7 activités RACI. À six mois, mesurer la charge effective et envisager la création d'un rôle Engineering Manager qui prendrait en charge le management RH des développeurs, laissant le TL sur les enjeux techniques.

IV.4.4 — Mesure de la santé d'équipe

Au-delà des indicateurs projet (velocity, taux d'engagement tenu, défauts en production), trois indicateurs de santé d'équipe sont suivis trimestriellement et présentés en rétro all-hands.

Indicateur	Mode de mesure	Cible
eNPS interne (recommandation de l'équipe à un pair)	Sondage anonyme trimestriel	≥ 50
Taux de présence en rituel	Compteur planning Scrum Master	≥ 90 %
Distribution de la charge de bug fix	Analyse tickets/membre/mois	Variation < 30 % moyenne
Taux de rotation annualisé	RH (départs / effectif moyen)	< 10 %
Satisfaction formation continue (4 axes)	Sondage annuel	≥ 4/5

IV.4.5 — Posture de leadership retenue

La posture de leadership défendue dans ce rapport est celle du servant leadership, formalisée par Robert Greenleaf en 1970 et reprise par Patrick Lencioni, Simon Sinek et la communauté agile internationale. Trois engagements structurants y sont attachés : (1) servir les conditions d'efficacité de l'équipe avant de servir l'image du leader, (2) reconnaître publiquement les contributions individuelles, en privé les erreurs, (3) protéger l'équipe des interférences hors-sprint (urgences fabriquées, demandes non priorisées du sponsor, polémiques externes).

Cette posture est exigeante. Elle suppose une vigilance permanente sur son propre rapport au pouvoir, une capacité à dire non avec respect, et une humilité sur la valeur de ses propres contributions. Elle me semble pourtant être la seule posture compatible avec la conviction profonde qui sous-tend ECOTRACK : un service public utile ne se construit pas contre les agents qui le portent, mais avec eux.

Conclusion générale et projection

1. Ce qui a été démontré

Ce rapport documente, sur plus de soixante pages, la conception, la réalisation et la projection d'industrialisation de la plateforme ECOTRACK — projet fil rouge mené en binôme sur l'année 2025-2026 dans le cadre du Mastère INGETIS, cursus M2 ITIS, filière Développement. Il étaye, bloc par bloc, l'acquisition des compétences certificatives attendues : planification d'un projet de développement logiciel, conception et développement d'une solution applicative, posture qualité-sécurité-maintenabilité, et pilotage d'une équipe de réalisation projetée.

Les quatre points qui nous paraissent les plus saillants au moment de remettre ce rapport :

- Sur le Bloc 1, la planification a tenu — au prix d'ajustements assumés (deux semaines de glissement maîtrisé, un livrable redécoupé, un livrable reporté avec contrepartie). Aucune disparition silencieuse de périmètre.
- Sur le Bloc 2, les sept modules métier sont opérationnels, testés, déployés. La stack technique est documentée, justifiée à chaque couche, défendable contre les alternatives écartées. Les tests automatisés (339 au total) couvrent 81 % des lignes du backend avec une stratégie pyramidale claire.
- Sur le Bloc 3, la posture qualité est outillée et mesurée. Aucune vulnérabilité HIGH ou MEDIUM ne subsiste au scan OWASP ZAP final. Les optimisations de performance ont été mesurées avant et après. La documentation est versionnée et testée empiriquement (onboarding ≤ 10 min validé deux fois).
- Sur le Bloc 4, l'organisation cible est dimensionnée, hiérarchisée, RACI-fiée. Les politiques d'inclusion, de gestion de conflit et de plan de formation sont formalisées avec un référentiel théorique nommé (CNV, servant leadership) et un budget chiffré.

2. Ce que ce parcours nous a appris

Au-delà des livrables, deux apprentissages nous semblent structurants pour la suite de nos trajectoires professionnelles.

Le premier porte sur la nature même du métier de développeur. Nous avons commencé ce parcours en pensant que notre travail consistait à produire du code qui fonctionne. Nous le terminons en sachant qu'il consiste à produire un système qui peut être opéré, corrigé, étendu, transmis et — éventuellement — arrêté par d'autres que nous. Le code n'est qu'un sous-produit. La vraie valeur réside dans la documentation des décisions (ADR), dans l'automatisation des contrôles (CI), dans la traçabilité des choix (Git), dans la lisibilité des intentions (code review). Ce déplacement de perspective n'est pas anodin : il transforme un développeur en ingénieur.

Le second porte sur l'articulation entre binôme et collectif. ECOTRACK a été mené à deux ; cela aurait pu suffire à considérer que la dimension humaine du Bloc 4 ne nous concernait qu'au futur. Le travail de structuration d'une équipe cible — fiches de poste, RACI, CNV, plan de formation, politique inclusion — nous a en réalité renvoyés à nos propres pratiques actuelles. Si nous ne savons pas expliquer notre architecture à un futur Tech Lead, c'est qu'elle n'est pas claire. Si nous ne savons pas formaliser ce qu'un

onboarding rapide réclame, c'est que nos assumptions implicites sont trop nombreuses. Le binôme, à sa propre échelle, nous a déjà fait éprouver ces questions : il a été un laboratoire des dynamiques que la Partie IV documente à plus grande échelle.

3. Compétences transversales

Les deux compétences transversales évaluées dans chaque bloc — anglais technique et numérique responsable — ont été traitées en fil rouge dans tout le rapport. Au moment de conclure, il est utile d'en restituer le bilan synthétique.

Anglais technique : la veille bibliographique a été conduite à 80 % sur des sources anglophones (RFC, blogs d'ingénierie, OWASP, ACM, conférences techniques). Les termes techniques sont employés en anglais sans francisation forcée quand ils n'ont pas d'équivalent stabilisé en français (CQRS, Event Sourcing, observability, idempotence, race condition). Les commentaires de code sont en anglais ; les ADR sont en français pour la dimension de défense. Une présentation orale en anglais du projet a été préparée en parallèle, exerçable sur sollicitation du jury.

Numérique responsable : la démarche d'éco-conception a été intégrée à chaque couche technique. Côté frontend : bundles minifiés, code splitting par route, lazy loading des bibliothèques lourdes (Leaflet), images au format AVIF, polices système prioritaires sur les polices web, audit Lighthouse régulier. Côté backend : pagination systématique, ETags pour le caching HTTP, compression gzip, requêtes batchées. Côté infrastructure : choix d'un VPS hébergé en France (OVHcloud Gravelines, refroidissement liquide, mix électrique faiblement carboné), conteneurs ARM possibles pour la consommation. Côté donnée : retention configurée (purge des télémétries brutes après 90 jours, conservation des agrégats). L'audit empreinte carbone simplifié (Boavizta) estime l'impact ECOTRACK à moins de 4 kg CO₂e par utilisateur actif et par an — comparable à un abonnement Netflix.

Au-delà de l'éco-conception applicative, ECOTRACK a une finalité environnementale par construction : la réduction du kilométrage de collecte de 25 % anticipée se traduirait, pour une collectivité de 500 000 habitants, par environ 240 000 km évités par an et 78 tonnes de CO₂e économisées — ordre de grandeur supérieur de plusieurs ordres à l'empreinte propre de la plateforme. Le numérique n'est responsable que lorsqu'il est en mesure de démontrer un solde net positif. Ce solde, ici, est défendable.

4. Projection : ce que nous ferions différemment

Trois inflexions nous semblent prioritaires pour une réédition de l'exercice.

- Démarrer la CI/CD au sprint 1. La leçon apprise au sprint 9 est trop chère. Une CI/CD imparfaite mais existante dès le début paie plus qu'une CI/CD ambitieuse mais retardée.
- Embarquer un utilisateur dès le sprint 3. Le projet a été conçu en autonomie binôme, avec des personas documentés mais non incarnés. Une heure d'entretien avec un vrai agent de collecte ou un vrai gestionnaire vaut mieux que 20 pages d'études de marché. Le coût d'opportunité de ne pas l'avoir fait est probablement le plus élevé du parcours.
- Documenter au fur et à mesure, pas en fin de bloc. La rédaction du présent rapport en J-2 du dépôt n'a été tenable que parce qu'une partie significative de la documentation avait été tenue à jour au fil

de l'eau (ADR, README, runbook). Pour les ADR qui ne l'ont pas été (ADR-008 à 012 reconstituées a posteriori), le coût rétrospectif est mesurable : le contexte décisionnel s'est partiellement effacé, certaines alternatives écartées ont dû être reconstituées par mémoire.

5. Mot final

ECOTRACK n'est pas, à ce stade, une plateforme déployée dans une vraie collectivité. C'est un projet école qui prouve que la chaîne de bout en bout peut être tenue par un binôme, qu'elle peut être documentée, sécurisée, mesurée, et qu'elle peut être transmise à une équipe. Le passage du projet école au produit industrialisé reste à faire ; il dépendra moins de la qualité du code livré que de la qualité des conversations qui auront lieu avec ses utilisateurs réels.

Ce qui nous reste de cette année, au-delà du code et des dossiers, c'est la conviction que le métier de développeur, pris au sérieux, est l'un des plus exigeants et des plus politiques qui soient : exigeant, parce qu'il réclame une articulation entre rigueur technique, sensibilité utilisateur et discipline organisationnelle ; politique, parce que chaque ligne de code arbitre une priorité, écarte une alternative, dessine en creux un avenir. Tenir ces deux exigences en même temps est un travail. Ce projet aura été notre manière, modeste mais sérieuse, de nous y entraîner.

Annexes

Annexe A — Captures d'écran de l'application en marche

Cette annexe regroupe, pour référence, l'ensemble des captures de l'application ECOTRACK en marche, qui illustrent les fonctionnalités présentées dans le corps du rapport. Ces captures ont été générées automatiquement par un test Playwright dédié (captures.spec.ts), garantissant leur reproductibilité.

Chaque capture a été prise en environnement réel (stack docker-compose locale, base seedée avec 240 capteurs, 4 rôles d'utilisateurs, jeu de données représentatif). Les références entre crochets renvoient aux figures du corps.

Capture	Fonctionnalité illustrée	Référence corps
01_login	Page de connexion, marque, comptes démo	Figure 3
02_dashboard_gestionnaire	Tableau de bord, KPI, cartographie Leaflet	Figure 5
03_optimisation_tournee	Optimisation TSP, arrêts ordonnés	Figure 6
04_agent_tournees	Interface agent, tournées du jour	Figure 7
05_citoyen_gamification	Espace citoyen, points, classement, défis	Figure 8
06_admin_users	Console admin, gestion des utilisateurs	Figure 9

Annexe B — Extraits de configuration CI/CD

Les extraits complets des workflows GitHub Actions (.github/workflows/ci.yml et .github/workflows/deploy.yml) sont versionnés dans le dépôt projet et illustrés au § II.4 (Figures 12, 13). Le détail des secrets attendus (GITHUB_TOKEN, DEPLOY_USER, DEPLOY_HOST, DEPLOY_KEY, REGISTRY_TOKEN) est documenté dans app/DEPLOY.md.

Annexe C — Métriques de qualité et performance

Les métriques détaillées présentées au Bloc 3 sont synthétisées ci-dessous pour relecture rapide.

Catégorie	Métrique clé	Valeur
Tests	Tests automatisés	339
Tests	Couverture lignes backend	81,2 %
Tests	Couverture lignes frontend	62,4 %

Tests	Taux flaky	0,8 %
Sécurité	Vulnérabilités HIGH actives	0
Sécurité	Findings OWASP ZAP critiques	0
Sécurité	Dépendances HIGH corrigées	3 / 3
Performance	Lighthouse moyen (Perf/A11y/BP/SEO)	95 / 98 / 100 / 100
Performance	p95 latence API (50 VU)	238 ms
Performance	Capacité MQTT (msg/h)	110 000 (× 13 nominal)
Performance	Gain index PostGIS	× 40
Performance	Gain cache Redis gamification	× 87
Documentation	ADR rédigées	23
Documentation	Endpoints documentés Swagger	47
Documentation	Délai onboarding mesuré	6–8 min

Annexe D — Schémas d'architecture détaillés

Le schéma de référence du système est présenté en Figure 1 (§ II.1.4). Trois compléments sont versionnés dans le dépôt projet pour relecture détaillée :

- livrables/dct/AppScreen_DCT.docx : Dossier Conception Technique de niveau ingénieur, 25 pages, couvrant l'architecture matérielle et logicielle dans le détail.
- livrables/dossiers/BLOC1/03_Architecture_Logicielle_ECOTRACK_DEV.docx : architecture en couches détaillée avec diagrammes UML (composants, déploiement, états).
- livrables/dossiers/BLOC1/04_Modelisation_UML_ECOTRACK_DEV.docx : cinq diagrammes UML (cas d'usage, classes, séquence, déploiement, états).

Annexe E — Glossaire et sources techniques

Glossaire

Terme	Définition
ADR	Architecture Decision Record : note formalisée traçant une décision d'architecture, son contexte et ses conséquences
CNV	Communication Non Violente, méthodologie d'expression et d'écoute formalisée par Marshall Rosenberg
CSP	Content-Security-Policy, en-tête HTTP qui restreint les sources autorisées de contenu
DAST	Dynamic Application Security Testing, analyse de sécurité dynamique exécutée contre l'application en marche
ECS	Compromis temps/qualité dans les heuristiques d'optimisation (2-opt vs Branch-and-Bound exact)
HSTS	HTTP Strict Transport Security, en-tête qui force le navigateur à utiliser HTTPS
LCP	Largest Contentful Paint, métrique Core Web Vitals mesurant le rendu de l'élément le plus volumineux visible
MQTT	Message Queue Telemetry Transport, protocole publish-subscribe léger utilisé pour l'IoT
OKR	Objectives and Key Results, cadre de pilotage par objectifs popularisé par Google
OWASP	Open Worldwide Application Security Project, organisation produisant le Top 10 des risques applicatifs
PostGIS	Extension PostgreSQL fournissant le support géo-spatial conforme OGC
RACI	Responsible, Accountable, Consulted, Informed : matrice de répartition des rôles par activité
RPO	Recovery Point Objective, durée maximale acceptable de perte de données après incident
RTO	Recovery Time Objective, durée maximale acceptable d'indisponibilité après incident
SAST	Static Application Security Testing, analyse de sécurité statique du code source
SLI / SLO	Service Level Indicator / Objective, métriques quantifiant la qualité de service rendue

Principales sources techniques anglophones consultées

- OWASP Top 10:2021 — <https://owasp.org/Top10/>
- NestJS Documentation — <https://docs.nestjs.com/>
- Prisma Documentation — <https://www.prisma.io/docs>
- MDN Web Docs (HTTP headers, CSP, Cookies) — <https://developer.mozilla.org/>
- PostGIS Manual — <https://postgis.net/documentation/>
- Eclipse Mosquitto Documentation — <https://mosquitto.org/documentation/>
- Caddy v2 Documentation — <https://caddyserver.com/docs/>
- k6 Documentation — <https://k6.io/docs/>
- Playwright Docs — <https://playwright.dev/docs/intro>
- Google web.dev (Core Web Vitals, Lighthouse) — <https://web.dev/>

Ouvrages et articles de référence

- Greenleaf, R. K. (1977). *Servant Leadership: A Journey into the Nature of Legitimate Power and Greatness*. Paulist Press.
- Rosenberg, M. (2003). *Nonviolent Communication: A Language of Life*. PuddleDancer Press.
- Nygard, M. T. (2011). *Documenting Architecture Decisions*. Cognitect.
- Forsgren, N., Humble, J., Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps. IT Revolution*.
- Lencioni, P. (2002). *The Five Dysfunctions of a Team*. Jossey-Bass.
- Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley.